SC2008 Computer Network

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Overview: History, Layers, and Encapsulation

> *"The Internet is not a single network, but a network of networks."* — Every photo you send, every video call, rides on a few simple ideas: layered abstractions, addresses, and encapsulation. This chapter builds those ideas from zero intuition to precise definitions.

From zero: no networking background assumed. We define every term from first principles—hosts, packets, protocols, delays, layers—with pictures before formalism. Intuition comes first, then the definition, then the example. The only prerequisites are SC1004 (vectors/matrices for later queueing intuition) and SC2000 (basic probability for later performance reasoning); nothing from them is assumed without explanation. Later chapters depend only on what is built here.

Learning Outcomes

After this chapter you will be able to:

(L1) describe the Internet as a network of networks and the role of hosts, routers, links, and protocols;

(L2) sketch the history from ARPANET to the modern Internet and explain packet switching versus circuit switching;

(L3) compare the OSI 7-layer and Internet 5-layer models and articulate service, interface, and protocol;

(L4) explain encapsulation and demultiplexing as a packet traverses the protocol stack;

(L5) compute end-to-end delay from transmission, propagation, processing, and queueing components.

1.1 What Is a Network? Intuition First

Think of sending a letter by post. You write the message, put it in an envelope with the destination address, drop it at a post office, and the postal network routes it hop by hop. The recipient opens the envelope and

reads the message. A computer network does the same, but the "envelope" is a *packet*, the "post offices" are *routers*, and the "roads" are *links* (copper, fibre, radio).

Formally, a network is a graph of *nodes* (hosts and routers) connected by *links*. A *host* (end system) runs applications—your laptop, phone, server. A *router* forwards packets toward their destination. An *Internet* (lowercase) is any network of networks; the *Internet* (capitalized) is the global public one governed by IETF standards. Communication obeys a *protocol*: a precise format plus rules for message exchange—exactly like postal conventions. Without a protocol, bits are just noise.

Definition 1.1 (Protocol and packet)**.** A *protocol* defines the format and order of messages and the actions taken on transmission and receipt. A *packet* is the unit of data that carries a payload plus headers added by protocols.

1.1.1 Addresses and Names

Every host needs an address, just as every house needs a postal address. On the Internet that address is an *IP address* (e.g., `155.69.0.1`); humans prefer *names* (e.g., `www.ntu.edu.sg`) because they are memorable. A directory service (DNS, Ch. 2) translates names to addresses. Within a host, a *port number* picks which application gets the packet—like an apartment number inside a building. Together, IP plus port name a socket endpoint uniquely.

Remark 1.1 (Why two levels?). IP gets the packet to the right *host*; the port gets it to the right *process* on that host. Without ports, a server could run only one application. With 16-bit ports (0–65535), many services share one machine: Web on 80, mail on 25, DNS on 53.

Two switching ideas compete for how the network shares links. *Circuit switching* (old telephone) reserves a whole path before talking—like booking an entire railway track. *Packet switching* chops data into packets that each find their way independently and share links statistically—like cars merging on a highway. The Internet chose packet switching because it uses links efficiently when traffic is bursty. Bursty means many idle gaps between bursts; reserving a whole circuit would leave it empty most of the time.

1.1.2 Delay, Loss, and Throughput

A packet suffers four delays on each hop: (i) *processing* (router examines header), (ii) *queueing* (waits behind earlier packets), (iii) *transmission* (L/R: L bits at link rate R bps), and (iv) *propagation* (d/s: distance d at speed $s \approx 2 \times 10^8$ m/s in fibre). Total nodal delay $d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$.

Listing 1.1: End-to-end delay: intuition to formula.

```
# Each hop adds four delays. Transmission is L/R, propagation is d/s.
def nodal_delay(L_bits, R_bps, d_m, s_mps=2e8, d_proc=0.001, d_queue=0.002):
    d_trans = L_bits / R_bps          # seconds to push bits onto link
    d_prop  = d_m / s_mps              # seconds to propagate
    return d_proc + d_queue + d_trans + d_prop

# Example: 1500-byte packet, 10 Mbps, 1000 km fibre
print(nodal_delay(1500*8, 10e6, 1e6)) # ~0.0092 s per hop
# Add hops and you get end-to-end delay.
```

Throughput is the bottleneck rate along the path: $\min_i R_i$. If your access link is 50 Mbps but the core is 10 Gbps, you get 50 Mbps. queueing makes delay random; loss happens when a router buffer overflows—a taste of SC2000 queueing theory we quantify later.

1.1.3 A First Example: Sending a Photo

Suppose you send a 8 Mbit photo from NTU to a friend in London via three hops. Each link is 20 Mbps and 3000 km. Transmission per hop is $8/20 = 0.4$ s; propagation per hop is $3 \times 10^6/2 \times 10^8 = 0.015$ s. Three hops give $\approx 3 \times (0.4 + 0.015) = 1.245$ s ignoring queueing—transmission dominates. For a 40-byte ACK, transmission is $320/20 \times 10^6 = 16$ μs and propagation dominates instead. This size-dependent crossover is why small packets feel "latency-bound" and large transfers feel "bandwidth-bound."

Remark 1.2 (Delay versus throughput). Students often conflate delay with throughput. A high-throughput fibre can still have large propagation delay (e.g., Singapore to London $\approx$ 70 ms). You can pour water faster (throughput) but water still takes time to travel the pipe (delay). Interactive voice needs low delay; bulk backup needs high throughput.

Try `traceroute` (or `ping`) to see hops and RTTs live: each line is one router, each time is $2 \times d_{\text{prop}} + d_{\text{queue}}$. Variation across pings reveals queueing.

1.1.4 Access, Core, and Edge

The edge is where hosts live (homes, phones, data centres). Access networks connect edge to core: residential fibre/DSL, WiFi, cellular. The core is the mesh of high-speed routers and fibres that interconnect access networks. Logically, the Internet is edge plus access plus core; physically, most bytes travel a few core hops. Content delivery networks (CDNs) push copies to the edge to cut delay—a preview of caching in Ch. 2.

1.2 A Brief History: From ARPANET to the Internet

Why did the Internet win? It stayed simple at the core and pushed complexity to the edges. That principle was learned incrementally.

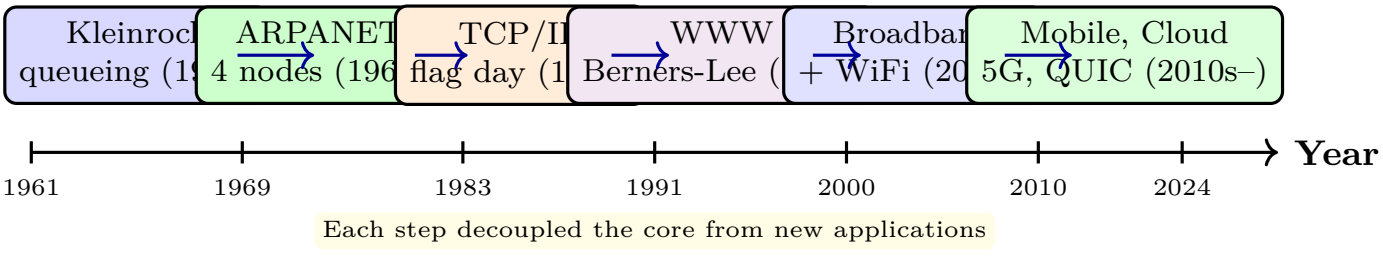


Figure 1.1: Internet timeline: theory → first packet-switched network → universal TCP/IP → World Wide Web → ubiquitous broadband and wireless. Each box keeps the core simple; innovation happens at the edges.

Key moments: 1961 Kleinrock used queueing theory (SC2000) to model packet delays. 1969 ARPANET linked four universities with packet switching. The 1970s produced TCP/IP—one narrow waist that any link technology could carry. 1983 was "flag day" when ARPANET switched permanently to TCP/IP. 1991 Tim

Berners-Lee added the Web on top, proving the layering idea: no change to routers was needed. The 2000s brought broadband and WiFi; the 2010s brought smartphones, clouds, and encrypted transports like QUIC. The pattern: a simple, universal core plus smarter edges.

1.2.1 Packet versus Circuit: A First Trade-off

Circuit switching guarantees rate but wastes capacity when the user is idle. Packet switching shares statistically: if 10 users each need 10 Mbps only 10% of the time, a 100 Mbps packet-switched link carries them with small queueing, whereas circuit switching would need $10 \times 10 = 100$ Mbps reserved anyway but leave 90% idle. The price is variable delay and occasional loss.

Example 1.1 (Why packet switching won)**.** A telephone call holds the circuit even during silence. Web browsing is silent 90% of the time between clicks. Packet switching lets hundreds of browsers share the same link and only contend when they actually send—statistical multiplexing. When contention is rare, queueing stays small and efficiency soars. When everyone clicks at once, queue builds and we need congestion control (Ch. 4).

1.3 Layered Architecture: Service, Interface, Protocol

Intuition first: imagine needing a document translated from English to Japanese via French. One translator does English↔French, another French↔Japanese. Each translator only talks to the adjacent layer and offers a *service* ("I translate French") via an *interface* ("give me French text"). How the translation is done internally is the *protocol* inside the layer. Networking stacks layers the same way: each layer offers a service to the one above and uses the service below.

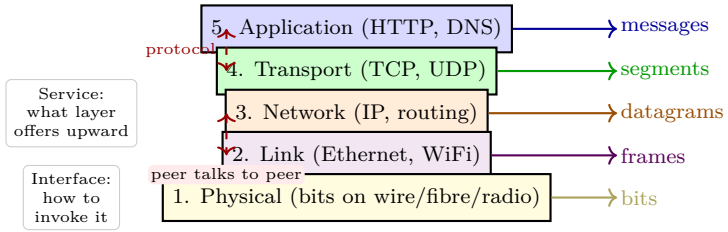


Figure 1.2: The Internet 5-layer stack. Each layer encapsulates the payload from above, adds its header, and uses the service below. The OSI 7-layer model splits Application into Application/Presentation/Session; TCP/IP merges them.

Definition 1.2 (Service, interface, protocol)**.** A *service* is what a layer offers to the layer above. An *interface* is how that service is invoked (e.g., socket API between Application and Transport). A *protocol* is the distributed algorithm peers at the same layer use to implement the service.

The Internet collapses OSI's seven layers into five: Application, Transport, Network, Link, Physical. Presentation and Session functions (encryption, compression) live inside the Application when needed. This minimal waist is why the same IP runs over fibre, WiFi, or satellite, and why HTTP, DNS, and streaming apps all share the same network—each app only interacts with Transport via sockets.

Why layering? Three benefits: (i) *modularity*—replace WiFi with Ethernet without touching TCP; (ii) *abstraction*—each layer hides complexity below; (iii) *reuse*—one Transport serves many applications. Cost:

headers add overhead and layering can hide useful information (e.g., wireless losses look like congestion to TCP).

Remark 1.3 (The hourglass). The Internet is an hourglass: many applications on top, many link technologies below, and a single narrow waist—IP—in the middle. Any app that speaks IP runs over any link that carries IP. That waist is why innovation at the edges (new apps) and at the bottom (new radios) can evolve independently.

1.3.1 Throughput Revisited: Bottlenecks

Think of a water pipe with three segments of diameters 10 cm, 2 cm, 10 cm: throughput is limited by the narrowest segment (2 cm), no matter how wide the others are. On the Internet the bottleneck is usually the access link (your home WiFi or phone), not the core. When many flows share a link, each gets a fair share—congestion control (Ch. 4) negotiates that share dynamically.

1.4 Encapsulation and Demultiplexing

When Host A sends "Hello" to Host B, the message descends the stack; each layer wraps it with a header (and sometimes a trailer) and passes the larger unit down. On receipt, each layer unwraps its header and demultiplexes upward.

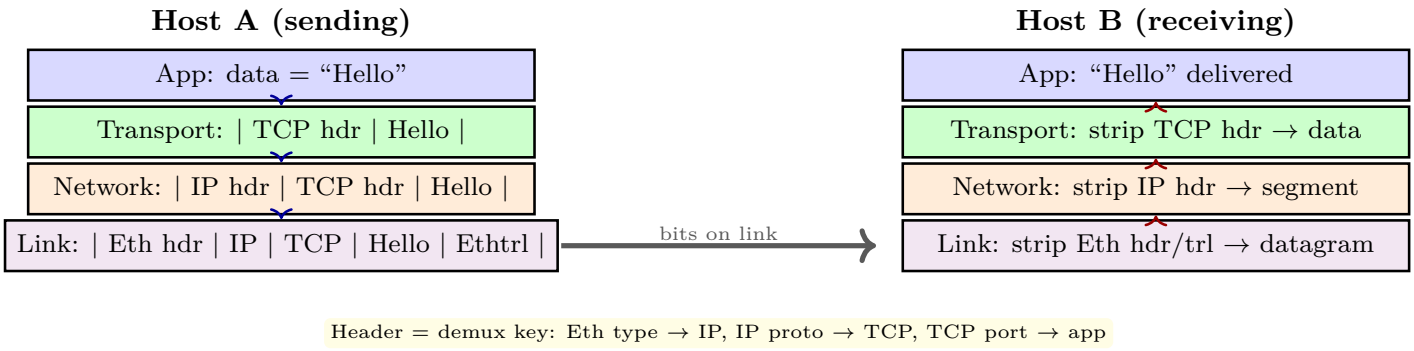


Figure 1.3: Encapsulation on sending and decapsulation on receiving. Each header names which upper-layer protocol should receive the payload (demultiplexing).

Demultiplexing uses header fields as keys: Ethernet's EtherType says "this is IP", IP's Protocol says "this is TCP", TCP's destination port says "deliver to the Web server, not the mail server." Without these keys, the receiver could not know which application should get the bytes.

Definition 1.3 (PDU and overhead)**.** Each layer names its packet: Application *message*, Transport *segment* (TCP) or *datagram* (UDP), Network *datagram*, Link *frame*. The header-to-payload ratio is overhead; for a 20-byte TCP header carrying 1460 bytes of data, overhead is $20/1480 \approx 1.35\%$—layering is cheap.

Listing 1.2: Layering as encapsulation: each layer wraps the payload.

```
class Layer:
    def __init__(self, name, header): self.name=name; self.header=header
    def encapsulate(self, payload): return f"|{self.header}|{payload}|"
    def decapsulate(self, packet): return packet.split("|")[2]  # strip one header

```

```
stack = [Layer("App",""), Layer("Transport","TCP"), Layer("Network","IP"), Layer("Link","ETH")]
payload = "Hello"
for layer in reversed(stack):  # top-down encapsulation
    if layer.header: payload = layer.encapsulate(payload)
print(payload)  # |ETH||IP||TCP|Hello|||
# Receiving reverses the order, each layer checking its header.
```

Example 1.2 (Ping as a layering lab)**.** Run `ping -c 3 www.google.com`: each ICMP echo is encapsulated as IP datagram, then Ethernet frame on WiFi. The reported time is $2(d_{\text{prop}} + d_{\text{queue}}) + d_{\text{proc}}$ round-trip. Try `ping -s 1000` versus `ping -s 100` to see transmission delay grow with packet size, and `traceroute` to see per-hop delays.

1.5 Chapter Notes

Layering follows the end-to-end principle (Saltzer–Reed–Clark, 1984). History from Leiner et al. *Brief History of the Internet* and Kurose–Ross. Delay taxonomy is standard: queueing analysis uses SC2000 probability in Ch. 3. Wireshark lets you see encapsulation live—capture a ping and peel headers.

1.6 Exercises

E1.1 **Circuit vs. packet.** Ten users each need 1 Mbps when active (active 20% of time, independent). A 4 Mbps circuit link reserves per active user. How many users can a circuit link support? What blocking probability would a packet link face with all ten sharing 4 Mbps? Why does bursty traffic favour packet switching?

E1.2 **Delay arithmetic.** A 12 000-bit packet crosses two links: $R_1 = 2$ Mbps, $R_2 = 5$ Mbps, each 500 km ($s = 2 \times 10^8$ m/s), $d_{\text{proc}} = 1$ ms, queueing negligible. Compute d_{trans} per link, d_{prop} per link, and total end-to-end delay. Which term dominates?

E1.3 **Layering trade-offs.** Your teammate proposes merging Transport and Network into one layer "to save headers." Give two modularity or reuse arguments against this and one scenario where cross-layer information would help. What invariant does layering protect?

E1.4 **Demultiplexing keys.** A host runs a Web server (TCP port 80), a DNS resolver (UDP port 53), and video streaming (UDP port 5004). When an IP datagram arrives with Protocol=6 (TCP) and dst port 80, trace the demultiplexing chain from Link to Application and state which header field is used at each step.

E1.5 **Wireshark peek.** Capture or sketch a ping (ICMP over IP over Ethernet). List the three headers you see, their sizes, and which field tells Ethernet that the payload is IP and which tells IP that the payload is ICMP.

Chapter 2

Application Layer: HTTP, DNS, and Sockets

"The Web is not the Internet." — The Internet moves bytes; applications give them meaning. HTTP lets you fetch a page, DNS translates names to addresses, and sockets are the door between your program and the network. This chapter builds all three from intuition to working code.

From zero: we build here, no sockets or Web background needed. We define every application concept from scratch—processes, client–server, messages, names, sockets—with everyday analogies before formalism. Intuition comes first, then the definition, then the wire format and Python code. The only prerequisites are SC1004/SC2000 as in Ch. 1; no prior networking or systems programming is assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish application architectures (client–server vs. peer-to-peer) and address a process via IP plus port;

(L2) describe HTTP message formats, methods, status codes, and persistent versus non-persistent connections;

(L3) explain DNS as a hierarchical, distributed database and trace iterative versus recursive resolution;

(L4) use the socket API to build simple UDP and TCP client–server programs in Python;

(L5) reason about cookies, Web caching, and conditional GET for performance.

2.1 Principles: Processes, Sockets, and Transport Services

An application is two *processes* (running programs) talking over the network. The Web's two processes are *browser* (client) and *server*; BitTorrent's processes are peers. To find a remote process you need its host's *IP address* and the process's *port number* (like a building address plus an apartment number). A *socket* is the

programming interface between the application and the transport layer—think of it as the door through which the app pushes messages out and receives messages in.

What does an app need from transport? Four questions: (i) reliable delivery? (file transfer yes, live voice can tolerate loss); (ii) throughput guarantee? (video needs a minimum rate); (iii) timing guarantee? (games need low delay); (iv) security? (now almost always via TLS). The Internet offers two transport services: *TCP* (reliable, ordered, congestion-controlled, connection-oriented) and *UDP* (unreliable, no frills, faster). Applications choose: HTTP, mail, and file transfer use TCP; DNS and streaming often use UDP.

Definition 2.1 (Client–server and P2P)**.** In *client–server*, the server has a well-known address and is always on; clients contact it and do not talk to each other directly. In *peer-to-peer*, peers are intermittently on and communicate directly with minimal server help, trading scalability for management complexity.

This is the end-to-end argument again: the network core (IP) stays simple; reliability, if needed, is implemented at the edges by TCP inside the end hosts.

Remark 2.1 (Choosing the right API). If you need every byte delivered (Web, mail, remote login), pick TCP and pay its handshake and congestion-control overhead. If you need speed and can handle loss yourself (DNS, VoIP, gaming), pick UDP and add only the reliability you need. HTTP/3 intentionally runs over QUIC (reliable) on top of UDP to combine both worlds.

2.2 The Web and HTTP

Intuition first: HTTP is like ordering at a restaurant. You (client) send a *request* ("GET /menu") and the kitchen (server) returns a *response* ("200 OK" plus the menu). The menu is an *object* (HTML file, image, video) addressed by a *URL*. Each object needs one fetch.

Two flavours of connection: *non-persistent* opens a fresh TCP connection per object and closes it—simple but pays TCP handshake per object. *Persistent* (default since HTTP/1.1) reuses one TCP connection for many objects, and *pipelining* can send the next request before the prior response arrives.

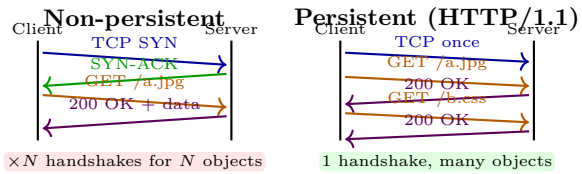


Figure 2.1: HTTP with non-persistent versus persistent TCP. Persistence amortises the handshake; pipelining overlaps requests. Time flows downward.

HTTP messages are plain text. A request has: request line (`GET /index.html HTTP/1.1`), headers (`Host:`, `Connection:`), blank line, optional body. A response has: status line (`HTTP/1.1 200 OK`), headers, body. Status families: `2xx` success, `3xx` redirect, `4xx` client error, `5xx` server error. Methods: `GET` fetches, `POST` submits, `PUT` stores, `DELETE` removes; `GET` is idempotent and cacheable.

Listing 2.1: Minimal HTTP exchange with sockets—what the browser does.

```
import socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("example.com", 80))  # TCP handshake happens here
```

```
req = "GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n"
s.sendall(req.encode())
resp = b""
while chunk := s.recv(4096):
    resp += chunk
print(resp[:600].decode(errors="ignore"))
s.close()
# Persistent would reuse s for multiple GETs before close.
```

Cookies let a stateless HTTP remember you: server sends `Set-Cookie`, browser stores it and replays on future requests. Cookies carry shopping carts, logins, and tracking—useful but privacy-sensitive. *Web caching* (proxy) keeps a copy closer to you; a conditional GET with `If-Modified-Since` returns `304 Not Modified` if cached copy is fresh, saving bandwidth.

Example 2.1 (Conditional GET saves bytes)**.** Your browser caches `logo.png` with `Last-Modified: Wed, 21 Aug 2026 07:00:00 GMT`. Next request sends `If-Modified-Since: Wed, 21 Aug 2026 07:00:00 GMT`. If the file is unchanged the server replies `304` with no body—a few hundred bytes instead of 50 KB. If modified, it replies `200 OK` with the new bytes.

2.2.1 HTTP Performance Tricks

Beyond persistence, HTTP uses *pipelining* (send next request before prior response arrives) and, in HTTP/2, *multiplexing* many streams over one connection. A *Content Delivery Network* replicates popular objects at edge caches nearer to you. Together they hide latency: the first fetch pays RTTs, later fetches hit the cache at LAN speed.

2.3 DNS: The Internet's Phone Book

You type `www.ntu.edu.sg`; the network needs an IP like `155.69.0.1`. *DNS* is the distributed, hierarchical database that translates names to addresses (and more). Why not one central table? A single server would be a single point of failure, a traffic bottleneck, and far from many clients.

Hierarchy first: the name `www.ntu.edu.sg` reads right to left—root (·), top-level domain (`sg`), second-level (`edu`), host (`www`). Each level is served by its own *authoritative* name servers. A *recursive resolver* (usually run by your ISP or 8.8.8.8) does the legwork; your host asks the resolver, which iteratively queries root → TLD → authoritative, caching answers along the way.

DNS records are the rows: `A` (name → IPv4), `AAAA` (IPv6), `NS` (who is authoritative), `CNAME` (alias), `MX` (mail). Messages normally ride over *UDP* (one query, one reply, fast); zone transfers use TCP. Caching and `TTL` (time to live) trade freshness for speed—exactly the consistency trade-off you will meet in databases (SC2207).

Listing 2.2: DNS lookup in two lines—and a manual UDP query.

```
import socket
print(socket.gethostbyname("www.ntu.edu.sg"))  # uses OS resolver

# Manual: crafting a minimal DNS query is possible with dnspython or raw bytes
```

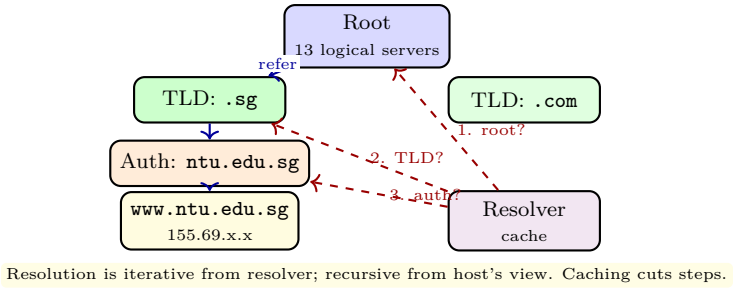


Figure 2.2: DNS hierarchy and resolution. The host asks its resolver once (recursive); the resolver walks the hierarchy iteratively. Each level delegates to the next via NS records.

```
# but gethostbyname shows the abstraction: name in, address out.
# Underneath it sent a UDP datagram to port 53 and parsed the A record.
```

2.3.1 DNS Performance and Security

DNS caching makes the Internet fast: once your resolver learns `ntu.edu.sg`, the next query for any host under it reuses the TLD and authoritative NS records. *TTL* trades speed for freshness—short TTL means faster updates but more queries. Security matters: vanilla DNS has no authentication, so an attacker can spoof replies; *DNSSEC* adds signatures so you can verify the answer really came from the authoritative server.

2.4 Socket Programming: Doors to the Network

A socket is the API between application and transport. You create one, bind or connect, then send/receive. The crucial difference: *UDP sockets* are connectionless—`sendto(address)` names the destination each time, like mailing a letter. *TCP sockets* are connection-oriented—`connect(address)` first, then `send/recv` like a phone call, with a separate socket per accepted client on the server.

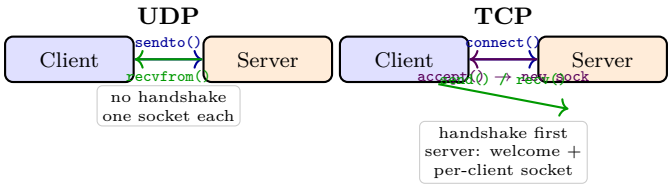


Figure 2.3: UDP versus TCP sockets. UDP is message-oriented and connectionless; TCP is byte-stream, connection-oriented, one new socket per client.

Listing 2.3: UDP echo and TCP echo—the two minimal client–servers.

```
# --- UDP server (connectionless) ---
import socket
us = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
us.bind(("", 12000))
while True:
    data, addr = us.recvfrom(2048)
    us.sendto(data.upper(), addr)   # echo uppercased

# --- UDP client ---
```

```
# cs = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# cs.sendto(b"hello", ("127.0.0.1", 12000))
# print(cs.recvfrom(2048)[0])

# --- TCP server (connection-oriented, one socket per client) ---
# vs = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# vs.bind(("", 12001)); vs.listen(1)
# while True:
#     conn, addr = vs.accept()      # new socket conn
#     data = conn.recv(1024)
#     conn.sendall(data.upper())
#     conn.close()
```

Pitfalls: TCP is a *byte stream*—one `send` may need multiple `recv`s; always loop. UDP datagrams are bounded (≈1472 bytes before IP fragmentation). Close sockets or you leak ports. Use `SO_REUSEADDR` during development so restarts do not hit "Address already in use."

Remark 2.2 (Stateless but not memoryless). HTTP itself remembers nothing between requests (stateless), yet cookies and server-side sessions add memory above it. This split keeps the protocol simple while letting applications be stateful—another instance of the end-to-end principle. Caches must be careful: a cached copy of a bank balance must not be served to another user; `Cache-Control: private` prevents that.

2.5 Chapter Notes

HTTP evolution: 1.0 (non-persistent), 1.1 (persistent, caching, cookies), 2 (multiplexed streams), 3 (QUIC over UDP). Fielding's REST dissertation formalised HTTP semantics. DNS is RFC 1034/1035; DNSSEC adds authentication. Sockets are Berkeley (1983); Beej's Guide remains the practical companion. Try Wireshark to watch HTTP and DNS live.

2.6 Exercises

E2.1 **HTTP timing.** A page has base HTML (4000 bytes) plus 4 images (each 8000 bytes). Non-persistent HTTP needs one TCP handshake (1 RTT) plus one request/response per object. Persistent reuses one handshake. If RTT= 20 ms and link rate is large, compare total time for both. How does pipelining help?

E2.2 **Status and methods.** A browser fetches `/a.html` and gets `301 Moved: Location /b.html`, then fetches `/b.html` and gets `200 OK` plus `Set-Cookie: id=7`. Explain each step, whether `GET` or `POST` is appropriate to submit a login form, and where the cookie is stored and replayed.

E2.3 **DNS walk.** Starting with an empty resolver cache, trace queries to resolve `mail.ntu.edu.sg`: which servers are contacted in order, what record types are returned, and where caching helps the next query for `www.ntu.edu.sg`? Contrast iterative versus recursive from the host's viewpoint.

E2.4 **Socket bytes vs. messages.** A TCP client does `send(b"HELLO")` once; the server loops `recv(2)` five times. What might each `recv` return, and why is this impossible with UDP? Write the correct TCP

receive loop that reassembles exactly 5 bytes.

E2.5 **Caching and consistency.** A Web cache stores `/data.json` with `TTL`=60 s. A client requests it at $t = 0, 30, 80$ s while the origin updates at $t = 50$ s. Which requests hit the cache and what do they return? How does `If-Modified-Since` with `304` reduce cost when TTL expires but the object has not changed?

Chapter 3

Transport Layer: UDP and Reliable Data Transfer

> *"The network promised to try its best, but trying is not delivering."* — Reliable transfer is the engineering answer to an unreliable world: we add numbers, timers, and retransmissions until loss, corruption, and delay become invisible to the application.

From zero: no transport background assumed. We start from the idea that the network layer delivers datagrams that may be lost or corrupted, then build up in steps: why ports exist, what UDP adds, and how to create reliability from scratch—acknowledgments, sequence numbers, timers—culminating in pipelined Go-Back-N. Pictures first, then formalism.

Learning Outcomes

After this chapter you will be able to:

(L1) explain multiplexing/demultiplexing and why transport needs port numbers;

(L2) describe the UDP segment format, checksum computation, and its use cases;

(L3) design stop-and-wait reliable transfer over a lossy/corrupt channel using sequence numbers, ACKs, and timers;

(L4) analyse Go-Back-N with its sliding window, cumulative ACKs, and utilization;

(L5) compare Go-Back-N versus Selective Repeat and compute efficiency.

3.1 Why a Transport Layer?

An application writes bytes; the network moves datagrams between hosts. The IP layer gets a packet *to the right house*, but which *person inside* should receive it? The transport layer adds *ports*—16-bit mailboxes—so

multiple processes on one host can share the network safely.

Intuition. Think of an apartment block (host) with one street address (IP). The lobby sorts letters by apartment number (port). Two apps—browser and DNS resolver—are two apartments sharing the same address. The operating system is the sorter.

Formally, *multiplexing* gathers data from sockets into segments at the sender; *demultiplexing* delivers arriving segments to the correct socket by destination port (and, for TCP, the 4-tuple). UDP demultiplexes by destination IP and port only; TCP uses source IP/port as well.

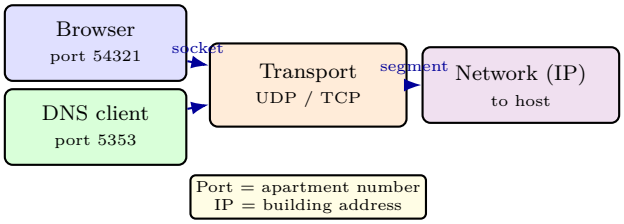


Figure 3.1: Demultiplexing intuition: IP gets the datagram to the host; port delivers it to the right socket. UDP keys on (dest IP, dest port).

3.2 UDP: Do Almost Nothing, Do It Fast

User Datagram Protocol adds exactly two things to IP: ports and an optional checksum. No handshake, no reliability, no ordering, no congestion control—just a thin wrapper.

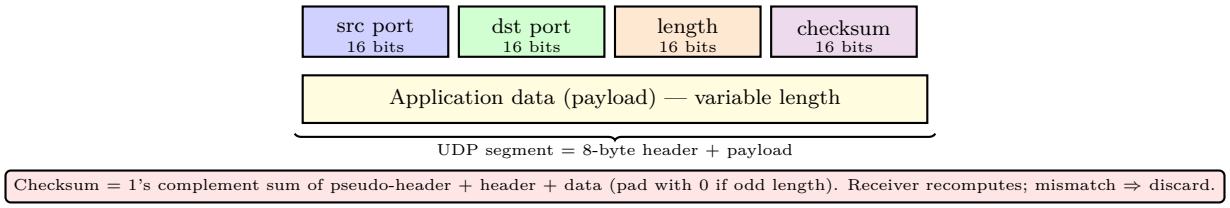


Figure 3.2: UDP segment format. Four 16-bit fields then payload. The simplicity is the point: minimal overhead, minimal guarantees.

Checksum. Sender sums 16-bit words (including a pseudo-header with IP addresses), takes ones' complement, and stores it. Receiver repeats the sum including the checksum—result `0xFFFF` means no detected error. It is a *detection* code, not correction. Formally,

$$\text{checksum} = \sim \sum_{\text{16-bit words}} w_i \quad \text{(ones' complement arithmetic)}. \tag{3.1}$$

If any bit flips, the sum changes with high probability, and the datagram is silently dropped.

When UDP? DNS lookups (one query, one reply; retries are cheap), DHCP, SNMP, QUIC's underlay, real-time media where a late retransmission is useless, and any protocol that implements its own reliability above UDP.

```
import socket
# UDP client: no connection, just sendto / recvfrom
```

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.sendto(b"hello udp", ("8.8.8.8", 53))
data, addr = sock.recvfrom(512)   # may be lost -- caller must retry
print(f"got {len(data)} bytes from {addr}")
# checksum is computed by OS/kernel; app sees only success or timeout
```

3.3 Reliable Data Transfer: Building Reliability from Nothing

The lower layer is *unreliable*: bits may flip, packets may be lost, order may shuffle, and delay may be unbounded. We successively strengthen the service.

rdt 1.0: reliable channel. Trivial—just send. Assumes perfection; useless in practice but fixes the API.

rdt 2.x: bit errors, no loss. Add *ACK* (positive) and *NAK* (negative). Sender waits for feedback. Problem: ACK/NAK can themselves be corrupted. Fix: number packets $0, 1$ alternating and ACK the sequence. rdt 2.1 uses ACK/NAK; rdt 2.2 uses ACKs only with duplicate-ACK meaning "please resend."

rdt 3.0: errors plus loss. Add a *countdown timer*. If no ACK arrives before timeout, retransmit. The receiver discards duplicates by checking the sequence number. This is *stop-and-wait*: one packet in flight at a time.

Definition 3.1 (Stop-and-wait invariant)**.** Sender has at most one unacknowledged packet with sequence $s \in \{0, 1\}$. Receiver expects $r \in \{0, 1\}$. On data with $s = r$, deliver and flip r; otherwise resend ACK_r.

Utilization reveals the flaw. Let L be bits per packet, R the link rate, RTT the round trip, and $d_{\text{trans}} = L/R$. Then

$$U_{\text{stop-wait}} = \frac{d_{\text{trans}}}{d_{\text{trans}} + RTT} = \frac{L/R}{L/R + RTT} \approx \frac{L}{R \cdot RTT} \quad \text{when } L/R \ll RTT. \tag{3.2}$$

For $L = 10^4$ bits, $R = 1$ Gbps, $RTT = 30$ ms, $U \approx 0.03\%$—the link idles while waiting.

```
def checksum16(data: bytes) -> int:
    """Ones' complement 16-bit checksum (UDP-style, simplified)."""
    if len(data) % 2 == 1:
        data += b"\x00"
    s = sum(int.from_bytes(data[i:i+2], "big") for i in range(0, len(data), 2))
    while s >> 16:           # fold carry
        s = (s & 0xFFFF) + (s >> 16)
    return (~s) & 0xFFFF

def rdt_send(sock, payload: bytes, addr, seq: int, timeout=0.5):
    """Stop-and-wait send with seq 0/1 and retransmission on timeout."""
    import socket as sk
    sock.settimeout(timeout)
    pkt = seq.to_bytes(1, "big") + payload
    while True:
        sock.sendto(pkt, addr)
```

```
        try:
            ack, _ = sock.recvfrom(1024)
            if ack and ack[0] == seq:    # correct ACK
                return seq ^ 1           # next sequence
        except sk.timeout:
            continue                     # retransmit
```

3.4 Pipelining and Go-Back-N

To fill the pipe, allow N unacknowledged packets. Sliding windows give three ideas: sequence numbers $0 \ldots 2^k - 1$, a *send window* $[base, base + N - 1]$, and *cumulative ACKs* (ACK n means "all $\leq n$ received").

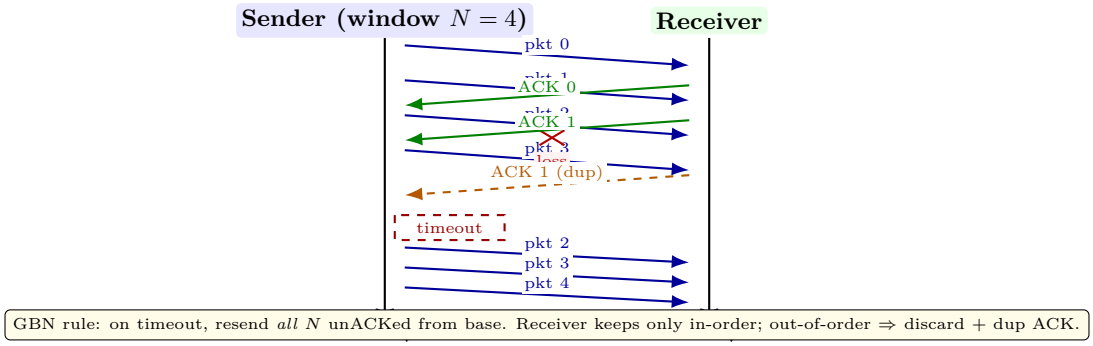


Figure 3.3: Go-Back-N ($N = 4$) time-line: pipelined sends, loss of pkt 2, cumulative ACKs, and timeout-driven retransmission of the whole window. Window slides only when base is ACKed.

Window mechanics. Sender keeps *base*, *nextseqnum*, and a timer for *base*. Receiver expects *expectedseqnum*. On correct in-order arrival, deliver and send ACK *expectedseqnum* then increment; otherwise discard and resend last ACK.

Utilization now improves roughly N-fold:

$$U_{\text{GBN}} = \min\left(1,\ \frac{N \cdot L}{R \cdot RTT}\right), \tag{3.3}$$

so choosing N to fill the bandwidth-delay product $R \cdot RTT$ keeps the pipe busy.

Selective Repeat contrast. GBN wastes retransmissions when only one packet in a large window was lost. *Selective Repeat* keeps per-packet timers and buffers out-of-order arrivals, retransmitting only what was lost. Cost: larger receiver buffer and k must satisfy $2^k > 2N$ to avoid aliasing, versus $2^k > N$ for GBN.

How large must the sequence space be? Example: $N = 4$ and $k = 2$ (space $0 \ldots 3$) alias under SR. Scenario: sender sends $0, 1, 2, 3$; all ACKed; window slides to $4 \equiv 0$, $5 \equiv 1 \ldots$ If ACKs are lost, sender resends 0 but receiver cannot tell old 0 from new 0—hence $2^k > 2N$. For GBN, $2^k > N$ suffices because the receiver never buffers out-of-order, so at most N aliases matter at once.

Remark 3.1 (Engineering takeaway). Stop-and-wait is Go-Back-N with $N = 1$. Pipelining bridges the gap

between utilization and complexity: N trades retransmission waste for throughput. The same bandwidth-delay insight reappears in TCP congestion control (Ch. 4), where *cwnd* is the adaptive N.

3.5 Chapter Notes

UDP's minimalism follows RFC 768 (Postel, 1980); reliable-transfer construction follows Kurose–Ross Ch. 3, itself rooted in Alternating Bit Protocol (Bartlett et al., 1969). Sliding-window analysis appears in every transport textbook; we revisit the bandwidth-delay product in Ch. 4 for TCP flow and congestion control.

3.6 Exercises

E3.1 **Ports and demultiplexing.** Host A runs two UDP clients with source ports 40001 and 40002 contacting the same DNS server (port 53). Describe the four values used for each direction and explain why the server's reply is not delivered to the wrong client. What would break if UDP omitted the checksum?

E3.2 **Checksum by hand.** Compute the ones' complement checksum of bytes `0x45 0x00 0x00 0x2C` viewed as two 16-bit words. Show the folded sum and final complement. Flip one bit in the second word and recompute; does the error go undetected?

E3.3 **Stop-and-wait utilization.** A 1 Gbps link with $RTT = 20$ ms carries 1500-byte packets. Compute $U_{\text{stop-wait}}$ and the throughput in Mbps. How large must N be for GBN to achieve $U \geq 0.9$?

E3.4 **GBN trace.** Window $N = 3$, sequence space $0 \ldots 7$. Sender sends 0,1,2; packet 0 is lost, ACKs never return. Enumerate each sender event (send, timeout, retransmit) for two full timeout cycles. Then repeat if packet 1 (not 0) had been the only loss and receiver uses cumulative ACKs.

E3.5 **GBN vs. Selective Repeat.** (a) Explain why GBN needs $2^k > N$ while Selective Repeat needs $2^k > 2N$ (give an aliasing scenario). (b) For $N = 4$, window loss $p = 0.2$, compare expected retransmitted packets per window under GBN versus SR.

Chapter 4

Transport Layer: TCP — Handshake, Flow, and Congestion Control

> *"If UDP shouts into the void, TCP calls, listens, and adapts."* — The same unreliable network can feel like a reliable byte pipe if both ends agree on numbers, windows, and how fast to probe. TCP is that agreement, tuned over forty years of congestion collapses.

From zero: built on Ch. 3. We assume you know ports, checksums, ACKs, and Go-Back-N. From there we add what a real operating system needs: a handshake to agree on parameters, a flow-control window to avoid drowning the receiver, and a congestion-control law to avoid drowning the network. Each mechanism is motivated by a failure story before its formalism.

Learning Outcomes

After this chapter you will be able to:

(L1) describe the TCP segment structure and establish/tear-down a connection via the three-way handshake;

(L2) explain sequence numbers, cumulative ACKs, and RTT estimation with timeout computation;

(L3) apply the flow-control invariant $LastByteSent - LastByteAcked \leq rwnd$ to reason about receiver protection;

(L4) analyse AIMD, slow start, and fast retransmit/fast recovery and sketch the sawtooth;

(L5) compare Reno/CUBIC congestion laws and identify when each dominates.

4.1 TCP at a Glance: A Reliable Byte Stream

TCP turns IP's datagram chaos into an ordered, reliable, bidirectional *byte stream*. Differences from UDP: connection-oriented, ordered, reliable, flow- and congestion-controlled, with a 20-byte (minimum) header full of

knobs.

Key fields (simplified): source/destination ports, 32-bit *sequence number* (byte index, not packet index), 32-bit *acknowledgment number* (next byte expected), 4-bit header length, flags (SYN, ACK, FIN, RST, PSH, URG), 16-bit *receive window rwnd*, 16-bit checksum, 16-bit urgent pointer, options (MSS, SACK, window scale). MSS is the max segment *payload*, negotiated via options.

Sequence numbers count bytes. If segment carries bytes 1000 . . . 1999, its sequence is 1000 and the ACK is 2000. This allows variable-size segments and precise retransmission of byte ranges—finer than GBN's packet numbers.

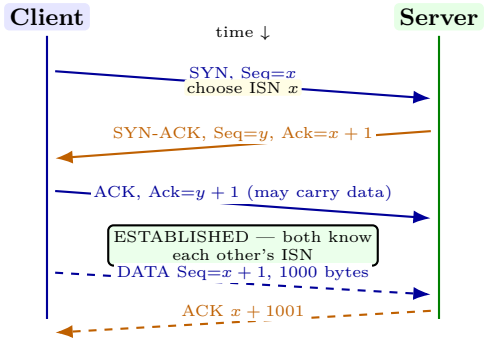


Figure 4.1: TCP three-way handshake. Each side picks an initial sequence number (ISN); the third ACK completes liveness. Data can piggyback on the third ACK and thereafter.

RTT estimation. TCP measures `SampleRTT` per segment and smooths:

$$\text{EstimatedRTT} \leftarrow (1-\alpha)\cdot\text{EstimatedRTT} + \alpha\cdot\text{SampleRTT}, \quad \alpha = 0.125, \tag{4.1}$$

$$\text{DevRTT} \leftarrow (1-\beta)\cdot\text{DevRTT} + \beta\cdot|\text{SampleRTT} - \text{EstimatedRTT}|, \quad \beta = 0.25, \tag{4.2}$$

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4\cdot\text{DevRTT}. \tag{4.3}$$

Karn's rule: ignore SampleRTT for retransmitted segments (ambiguity). This is TCP's timer from Ch. 3 made adaptive.

```python
import socket
# TCP client vs UDP: connect, stream, close -- reliability is inside TCP
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect(("example.com", 80))          # triggers 3-way handshake
    s.sendall(b"GET / HTTP/1.0\r\n\r\n")    # byte stream, not datagrams
    data = s.recv(4096)                     # ordered, no loss seen by app
print(data[:80])
# sock.send / recv boundaries are NOT preserved -- TCP is a byte pipe
```

4.2 Flow Control: Not Overwhelming the Receiver

Even if the network is fine, a fast sender can overrun a slow receiver that has only $RcvBuffer$ bytes. Flow control enforces an invariant.

Intuition. Receiver advertises *how much spare room remains*: $rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$. Sender obeys $LastByteSent - LastByteAcked \le rwnd$. When $rwnd = 0$, sender probes with 1-byte segments.

Formally, maintain variables $LastByteSent$, $LastByteAcked$, $rwnd$ (from last ACK). Before sending, require $LastByteSent+len \le LastByteAcked+rwnd$. Window field in the header carries $rwnd$ (scaled by window-scale option for high-BDP paths).

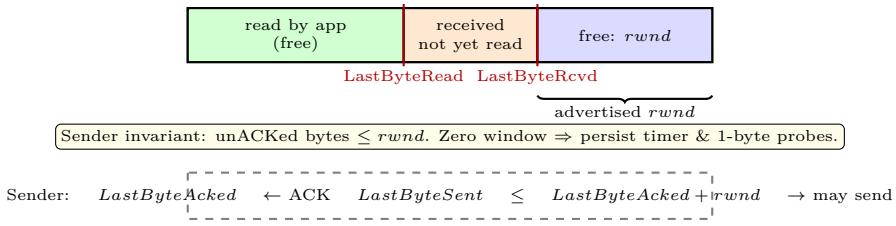


Figure 4.2: Flow-control window. The receiver advertises remaining buffer; the sender never exceeds it. Flow control protects the receiver, not the network (that is congestion control).

Flow control is receiver-side; *congestion* control below is network-side. Confusing them is the classic exam trap.

4.3 Congestion Control: Not Overwhelming the Network

When too many senders push too fast, queues overflow, loss rises, and goodput collapses (1986 Internet collapse). Congestion control makes each sender *probe* for bandwidth and *back off* on signal (loss or ECN). State variable *cwnd* (congestion window, in MSS) limits unACKed bytes alongside *rwnd*:

$$\text{effective window} = \min(cwnd,\ rwnd). \tag{4.4}$$

AIMD and the sawtooth. With no loss, increase linearly; on loss, cut multiplicatively. Reno specifics: *slow start* doubles each RTT ($cwnd \leftarrow cwnd + MSS$ per ACK, so exponential), then *congestion avoidance* grows by $MSS/cwnd$ per ACK ($\approx +1$ MSS per RTT), and on 3 duplicate ACKs halve *cwnd* (fast retransmit/recovery). Timeout collapses *cwnd* to 1 MSS and sets $ssthresh = cwnd/2$.

Why AIMD works: Chiu–Jain (1989) showed additive increase probes fairly, multiplicative decrease converges to fairness and efficiency from any start—geometry, not luck. Fairness means n flows sharing a bottleneck converge to equal rates; efficiency means the sum tracks capacity.

CUBIC. Default in Linux since 2006, CUBIC replaces RTT-dependent linear growth with a cubic function of *time since last loss*:

$$W(t) = C\,(t-K)^3 + W_{\max}, \quad K = \sqrt[3]{W_{\max}\beta/C}, \quad \beta \approx 0.7,\ C = 0.4, \tag{4.5}$$

where $W_{\max}$ is *cwnd* before the last reduction. After a loss, CUBIC drops to $\beta W_{\max}$, then grows concavely to $W_{\max}$, plateaus, and probes convexly beyond. This decouples growth from RTT (fairer across heterogeneous paths) and scales better on high-BDP links where Reno's +1/RTT is too timid.

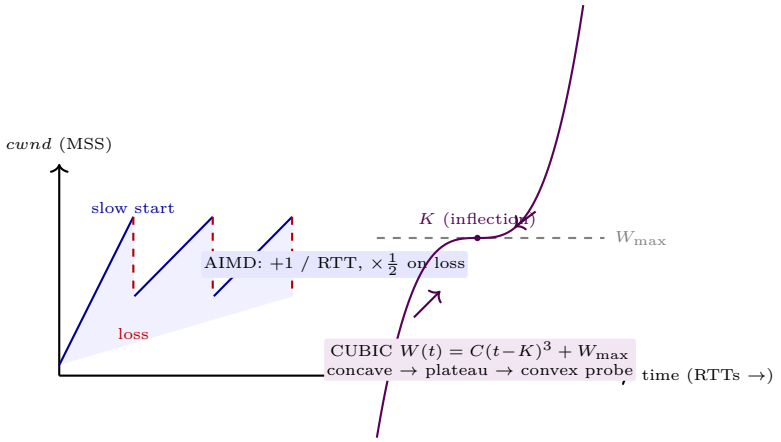


Figure 4.3: Left: AIMD sawtooth (Reno) — linear rise, multiplicative halving. Right: CUBIC's cubic probe around $W_{\max}$, friendlier to high-BDP paths and less RTT-biased.

```python
def aimd_simulation(steps=40, init=1, ssthresh=16):
    """Tiny AIMD/Reno sketch: slow-start then +1/RTT, halve on loss."""
    cwnd, sst = float(init), ssthresh
    trace = []
    for t in range(steps):
        loss = (t % 12 == 11)          # synthetic periodic loss
        if loss:
            sst = max(cwnd/2, 2)
            cwnd = sst if t > 8 else 1  # fast-recovery vs timeout
        else:
            if cwnd < sst:             # slow start: double per RTT (approx)
                cwnd *= 2
            else:                      # congestion avoidance: +1 per RTT
                cwnd += 1
        trace.append(round(cwnd, 1))
    return trace

print(aimd_simulation()[:12])
# CUBIC: W(t)=C*(t-K)^3+Wmax  with K=cuberoot(Wmax*beta/C)
def cubic_W(t, Wmax, C=0.4, beta=0.7):
    import math
    K = (Wmax*beta/C)**(1/3)
    return C*(t-K)**3 + Wmax
```

Fairness note: Reno favours low-RTT flows (they clock faster); CUBIC is closer to RTT-fair because its clock is real time. BBR (2016) departs further, modelling bottleneck bandwidth and RTT rather than reacting only to loss—beyond our scope but the natural next step.

Teardown and the full lifecycle. TCP closes with FIN flags: each side sends FIN when done, the peer ACKs, and the final ACK is followed by a 2×MSL TIME-WAIT (typically 60 s). TIME-WAIT lets delayed duplicates expire and ensures the final ACK was received; without it, a new incarnation of the same 4-tuple could be polluted by old bytes. Fast recovery, SACK blocks, and ECN marks all refine the loss signal without leaving this core loop.

Remark 4.1 (SACK and ECN: modern loss signals). Selective ACK (SACK, RFC 2018) lets the receiver report

non-contiguous blocks, so the sender retransmits only holes rather than the whole window—like SR improving on GBN. Explicit Congestion Notification (ECN, RFC 3168) lets routers mark rather than drop, converting loss into an early warning. Both are negotiated during the handshake via TCP options.

4.4 Chapter Notes

TCP specified in RFC 793 (1981) with congestion control added after Jacobson (1988). Stevens' illustrations and Peterson–Davie remain definitive. AIMD fairness is Chiu–Jain (1989); CUBIC is Ha–Rhee–Xu (2008), Linux default. Fast retransmit, SACK (RFC 2018), and ECN (RFC 3168) refine the core loop presented here.

4.5 Exercises

E4.1 **Handshake and ISN.** (a) Why does each SYN carry an ISN rather than starting at 0? (b) Trace establishment when client's SYN is duplicated and delayed: show why 2 messages are insufficient and the third ACK fixes ghost connections. (c) What does simultaneous open look like?

E4.2 **RTT and timeout.** SampleRTTs: 100, 120, 90 ms. Start EstimatedRTT=100, DevRTT=10, $\alpha = 0.125$, $\beta = 0.25$. Compute EstimatedRTT, DevRTT, and TimeoutInterval after each sample. Why does Karn's algorithm ignore SampleRTT for retransmitted segments?

E4.3 **Flow control invariant.** RcvBuffer=8000 B, LastByteRead=2000, LastByteRcvd=6000. Compute *rwnd*. If sender has LastByteSent=6200 and LastByteAcked=3000, can it send a 1000-B segment? What happens at $rwnd = 0$ and how does the persist timer help?

E4.4 **AIMD sawtooth.** MSS=1000 B, $ssthresh = 16$ MSS, $cwnd = 1$ MSS. No loss for 6 RTTs with Reno rules (slow start to *ssthresh*, then +1/RTT). Give *cwnd* each RTT. At RTT 7, triple duplicate ACK halves *cwnd*. Give new *ssthresh* and *cwnd* under fast recovery, then next RTT's *cwnd*.

E4.5 **CUBIC vs. Reno.** $W_{\max} = 40$ MSS, $\beta = 0.7$, $C = 0.4$. Compute K and $W(K + 2)$ for CUBIC. For a 100 Mbps $\times$ 100 ms BDP path with 1500-B MSS, argue why Reno's +1/RTT would take many minutes to recover while CUBIC recovers in seconds.

Chapter 5

Network Layer: IP, Forwarding and Routing

"The network layer glues billions of links into one Internet." — IP promises only best-effort delivery; routing makes the promise useful by finding paths at planetary scale.

From zero: we build here, no prior networking assumed. We start from "how does a packet cross many networks?" with pictures before headers or algorithms. Every notion—datagram, prefix, forwarding table, shortest path, autonomous system—is defined on first use. No IP or graph-theory background is needed beyond basic sets.

Learning Outcomes

After this chapter you will be able to:

(L1) describe the IPv4 datagram format, addressing, CIDR and subnetting;

(L2) distinguish forwarding (data plane) from routing (control plane) and apply longest-prefix match;

(L3) execute Dijkstra's link-state algorithm and prove its correctness invariant;

(L4) compare link-state vs. distance-vector (Bellman–Ford) and count-to-infinity;

(L5) explain BGP as path-vector inter-domain routing: ASes, peering, and policy.

5.1 Why a Network Layer?

Transport gives end-to-end reliability between hosts, but who moves a packet *across* many networks? The network layer does. Think of a postal system: you write a destination address on the envelope (IP address); each post office (router) looks only at the address and forwards it toward the destination. The sender need not know the full route. Two planes: *data plane* (per-packet forwarding in hardware, nanoseconds) and *control*

plane (computing forwarding tables, milliseconds to minutes). IP is the narrow waist—everything over IP, IP over everything. Without a network layer, every pair of networks would need a custom gateway; with IP, N networks need only N IP interfaces, not N^2 translators.

5.2 IPv4 Datagram and Addresses

An IPv4 datagram carries a 20-byte header (plus options) and payload.

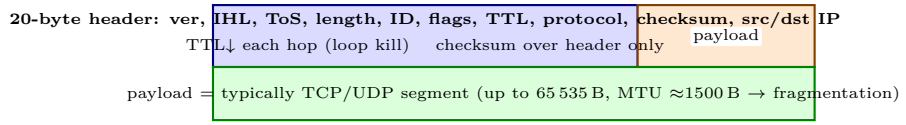


Fig: IPv4 datagram. Header contains everything a router needs to forward.

Addresses are 32-bit, written as dotted decimal: 192.168.1.10 = 11000000.10101000.00000001.00001010. Originally classful (A/B/C); now *CIDR*: $a.b.c.d/x$ means first x bits are network prefix, rest host. Example: 203.0.113.0/24 leaves $2^8 = 256$ addresses. A host with /26 has subnet mask 255.255.255.192. Longest-prefix matching (next section) makes CIDR aggregation possible—ISP advertises one /16 instead of 256 /24s. IPv6 fixes exhaustion with 128-bit addresses (hex, e.g. `2001:db8::1`), same forwarding idea.

Definition 5.1 (Subnet)**.** A *subnet* is a set of interfaces with a common prefix whose packets can be delivered without a router (shared link or switched LAN). A host's address has network part (prefix) and host part.

CIDR aggregation: an ISP holding 14.24.0.0/15 covers $2^{17} = 131\,072$ addresses—range 14.24.0.0 to 14.25.255.255, i.e. two adjacent /16s. Without aggregation the global BGP table would exceed 800k entries; with CIDR it stays manageable. On a host, /26 means 64 addresses (62 usable) with mask 255.255.255.192. Fragmentation: if payload > MTU, router/host splits into fragments sharing the same ID. Example: 4000 B datagram over MTU 1500 → fragments 1480+1480+1040 payload (20 B header each, offset in 8 B units, MF flag). Reassembly only at destination—avoided today via Path MTU Discovery (sender probes with DF bit, learns bottleneck MTU). IPv6 forbids router fragmentation entirely. Header fields matter: TTL (now hop limit) prevents loops—each router decrements, drops at zero and sends ICMP Time Exceeded (basis of `traceroute`). Protocol field demultiplexes to TCP (6), UDP (17), etc. Header checksum is recomputed each hop because TTL changes; IPv6 drops it (link layers already check).

5.3 Forwarding: Longest-Prefix Match

Each router stores a *forwarding table*: (prefix, link) pairs computed by routing protocols. On arrival, it finds the *longest* prefix matching the destination IP and forwards on that link. Default route 0.0.0.0/0 catches the rest. Forwarding is per-datagram, stateless, and must be fast—core routers handle > 100M lookups/s via TCAM or trie in ASIC. This is why aggregation works: a less-specific route covers many destinations, overridden where needed by more specific ones. A binary trie on prefix bits implements LPM in software: walk bits from MSB, remember last matching prefix. Hardware TCAM matches all entries in parallel.

```
1 def longest_prefix_match(dst_ip, table):
```

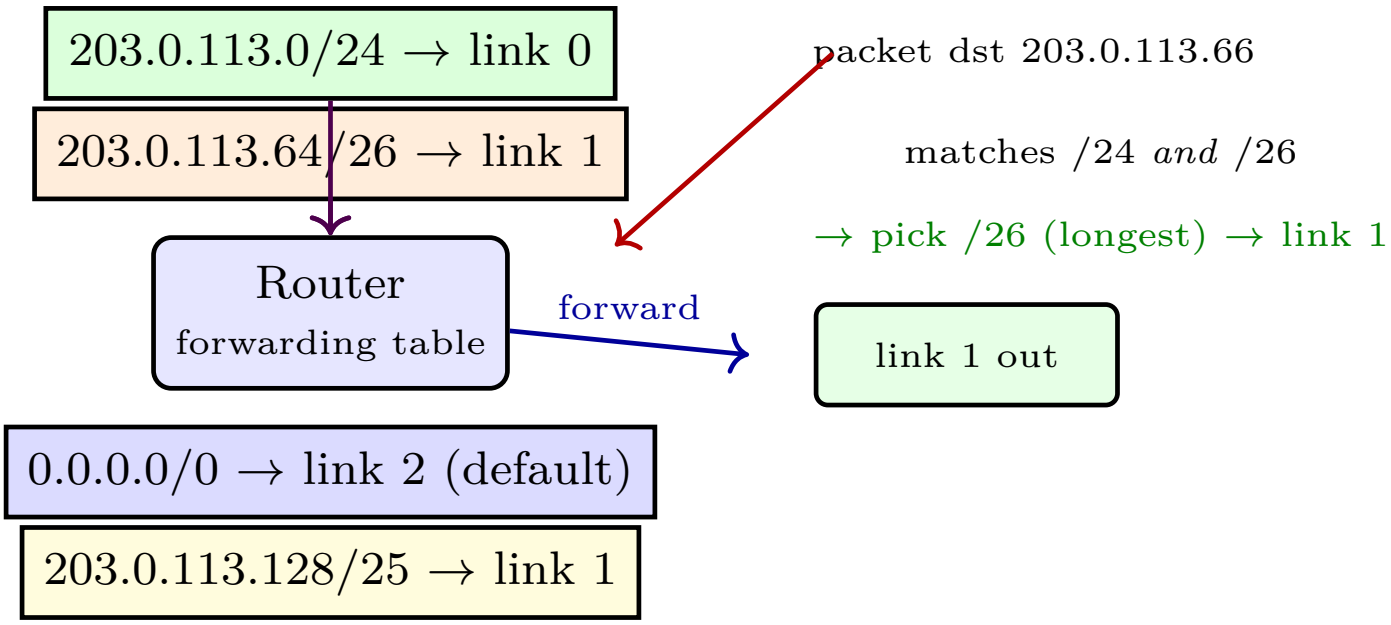


Figure 5.1: Longest-prefix match: 203.0.113.66 matches both /24 and /26; the more specific /26 wins. Table lookups are done in hardware (TCAM) for millions of prefixes.

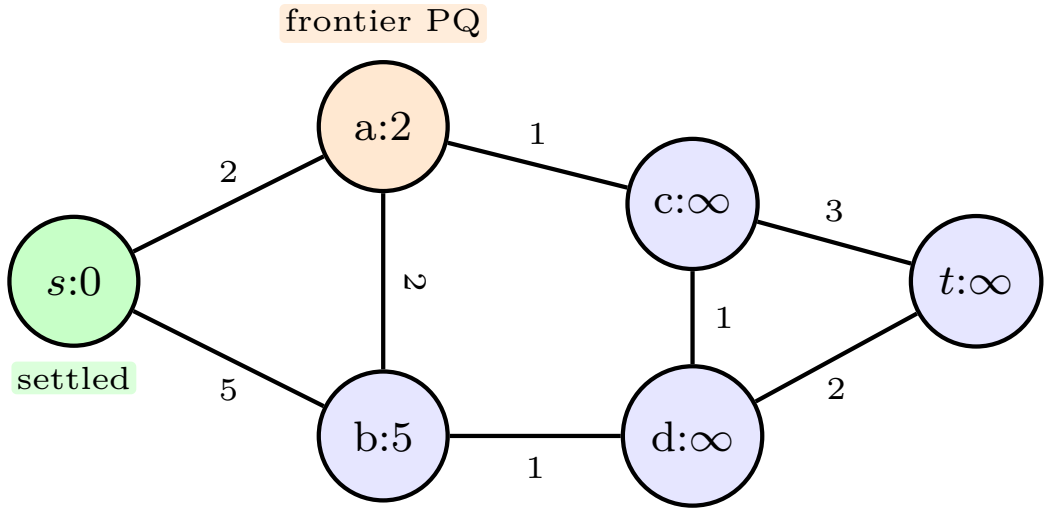
```
    """table: list of (prefix_int, mask_len, link). dst_ip: 32-bit int."""
    best_link, best_len = None, -1
    for pref, mlen, link in table:
        mask = ((1 << 32) - 1) ^ ((1 << (32 - mlen)) - 1) if mlen>0 else 0
        if (dst_ip & mask) == (pref & mask) and mlen > best_len:
            best_len, best_link = mlen, link
    return best_link

# Example: 203.0.113.66 = 0xCB007142
table = [(0xCB007100,24,0),(0xCB007140,26,1),(0,0,2),(0xCB007180,25,1)]
print(longest_prefix_match(0xCB007142, table))  # -> 1 (the /26)
print(longest_prefix_match(0xCB007190, table))  # -> 1 (the /25)
```

Forwarding vs. routing: forwarding is the *act* (table lookup per packet); routing is the *computation* (building that table via Dijkstra/BGP). Separation lets forwarding stay simple and fast while routing can be complex and distributed.

5.4 Dijkstra: Link-State Routing

How does the control plane *fill* those tables? Each router must know shortest paths. Model the network as a weighted graph $G = (V, E)$, $c(u, v) \geq 0$. Link-state protocols (OSPF) flood each link's cost to all routers, so every router learns the full graph and runs Dijkstra locally. *Intuition first.* Stand at source s and watch a wavefront expand at speed $1/c$. The first time the wave hits a node, it arrived via the shortest path—never improve later because all edges are non-negative. Dijkstra simulates this wavefront with a priority queue.

```
import heapq
def dijkstra(adj, s):
    """adj[u]=list of (v,w) w>=0. Returns dist, parent."""
    n = len(adj); INF = 10**18
```



Dijkstra wavefront: green settled, orange in PQ, distances are current best.

Figure 5.2: Dijkstra from s after settling s: frontier $\{a(2), b(5)\}$. Next extract a, relax its edges. Edge weights ≥ 0 guarantee settling is final.

```
    dist = [INF]*n; parent = [-1]*n
    dist[s]=0; pq=[(0,s)]
    visited=[False]*n
    while pq:
        d,u = heapq.heappop(pq)
        if visited[u]: continue
        visited[u]=True          # settled: d is final shortest distance
        for v,w in adj[u]:
            if dist[v] > d + w:  # relax
                dist[v]=d+w; parent[v]=u
                heapq.heappush(pq,(dist[v],v))
    return dist, parent

adj=[[(1,2),(2,5)],[(2,2),(3,1)],[(4,1)],[(4,3),(5,2)],[(5,1)],[]]
print(dijkstra(adj,0)[0])  # [0,2,4,3,5,6]
```

Invariant: when u is extracted, $dist[u]$ equals the true shortest-path distance $\delta(s, u)$. Proof by cut: any alternative path must cross from settled S to $V \setminus S$ with cost $\geq$ min frontier, so cannot beat u. The $w \geq 0$ condition is essential—negative edges could offer a shortcut through unsettled nodes. Complexity $O((n + m) \log n)$ with binary heap; $O(m + n \log n)$ with Fibonacci heap. OSPF floods link states via LSAs so every router runs Dijkstra independently and consistently.

5.5 Distance-Vector and BGP

Distance-vector (Bellman–Ford) is dual: each node x keeps $D_x(y) = \min_v\{c(x, v) + D_v(y)\}$ and exchanges vectors with neighbours. No global view; iterates to convergence in $O(nm)$ worst case. It underlies RIP (hop count, max 15). *Pitfall:* link cost increase can cause count-to-infinity. Example: a–b–c line ($c = 1$ each). If b–c fails, b thinks it can reach c via a (which routes through b!) and both increment slowly to infinity. Poisoned reverse (advertise ∞ back to next-hop) mitigates for two-node loops but fails on larger cycles—hence RIP

limits diameter. For Internet scale, no router can run Dijkstra on 100k networks. The Internet is federated into *Autonomous Systems* (ASes—ISPs, campuses). *BGP* (Border Gateway Protocol) is path-vector: routers exchange *entire paths* (AS sequence), not just distances, so loops are detected by seeing own AS in path. BGP is *policy*-driven: an AS may prefer a longer customer route over a shorter peer route (economics beats

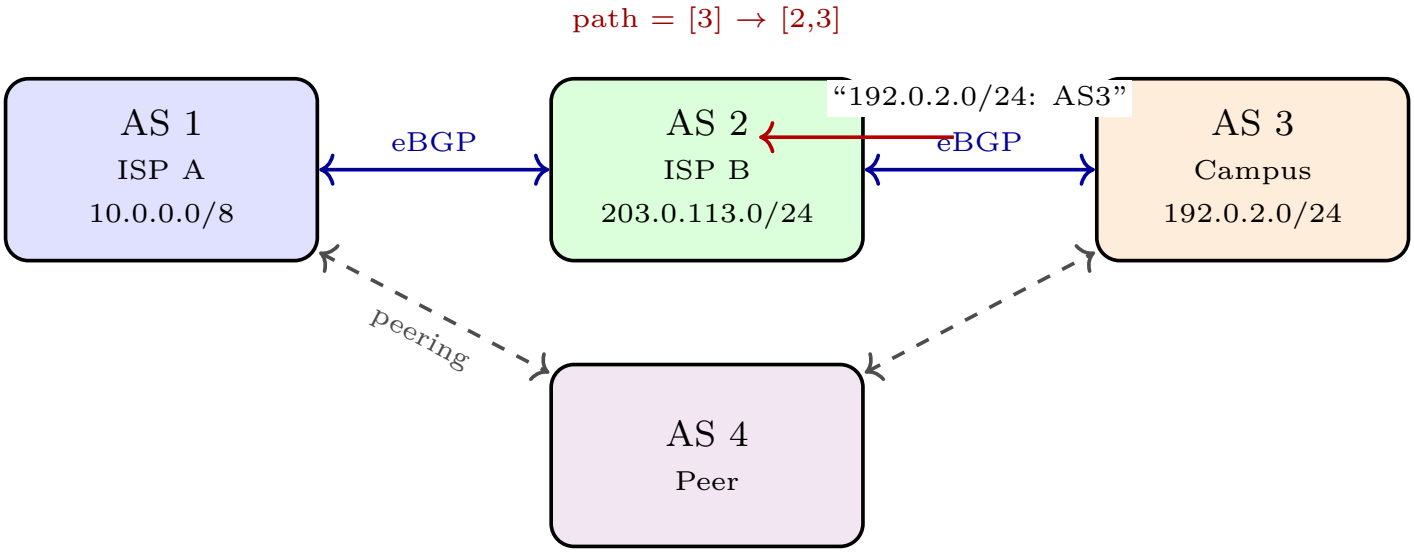


Figure 5.3: BGP inter-domain routing: each AS advertises its prefixes with AS-path. Policy (prefer customer > peer > provider) overrides shortest path; hot-potato forwards to nearest egress.

hops). Decision steps: (1) highest local-pref (operator policy), (2) shortest AS-path, (3) lowest MED (hint from neighbour), (4) hot-potato (lowest IGP cost to egress), (5) tie-break router ID. Hot-potato forwards to the closest egress point, even if the AS-path is slightly longer internally. Inside an AS, iBGP disseminates externally learnt routes and the IGP (OSPF) handles internal hops. Convergence is slow—path exploration after failures can take minutes and cause transient loops. BGP security (prefix hijacks) is a real risk; RPKI/ROA helps authenticate origins.

5.6 NAT: One Public Address for a Whole Home

Intuition first. A home router is like an office receptionist with one public phone number: inside, every desk has a short extension (private address); outside callers only see the reception number, and the receptionist routes by extension (port). IPv4 has only $2^{32} \approx 4.3$ billion addresses, far too few for every device, so RFC 1918 reserves private ranges—10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16—reusable inside each network but never routed on the public Internet. Your laptop at 192.168.1.10 keeps that address at home, at NTU, and at a cafe simultaneously; a NAT gateway at the edge translates it to the one globally routable public address the ISP assigned. NAPT (NAT with port translation, what home routers do) multiplexes many private hosts onto that single public IP by rewriting the source port. Outgoing: (192.168.1.10:5001 → 142.250.1.1:80) becomes (203.0.113.7:61001 → 142.250.1.1:80), and the gateway records (61001 ↔ 192.168.1.10:5001) in its translation table; a second laptop using the same private port gets a different public port (say 61002), so the 16-bit port field supplies ∼64k concurrent flows per public IP. Incoming replies to 203.0.113.7:61001 are reverse-mapped and forwarded inside; unsolicited inbound packets with no table entry are dropped. Drawback: NAT breaks the end-to-end principle—

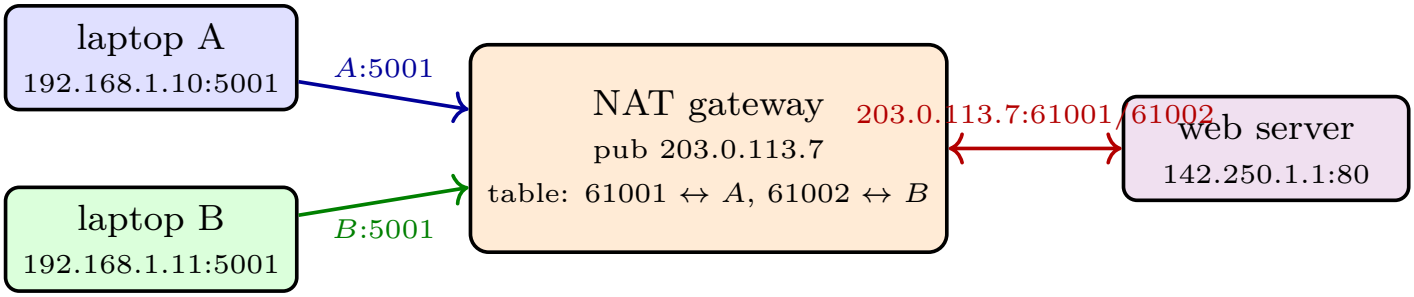


Figure 5.4: NAPT multiplexes two hosts sharing private port 5001 onto one public IP via distinct public ports. Return traffic is demultiplexed by table lookup; unsolicited inbound has no entry and is dropped.

a host behind NAT has no stable public address, so inbound connections (P2P calls, game hosting, servers) fail without hole-punching (STUN/TURN/ICE), UPnP port-forwarding, or manual rules. Carrier-grade NAT stacks a second translation at the ISP, adding latency and logging burden (abuse traceback needs timestamped port logs). NAT also mangles protocols embedding addresses (classic FTP, SIP), needing application gateways. It was a survival fix for exhaustion, not architecture: the long-term answer is IPv6.

Example 5.1 (NAPT walk-through)**.** Laptops $A = 192.168.1.10{:}5001$ and $B = 192.168.1.11{:}5001$ both open `142.250.1.1:80`. Gateway (public 203.0.113.7) assigns $61001 \leftrightarrow A$, $61002 \leftrightarrow B$ and rewrites sources. Server replies to 203.0.113.7:61001 and 203.0.113.7:61002; the gateway maps each back to the right laptop. Netstat on A still shows 5001—translation is invisible to endpoints. If the table holds 60k entries and each web page opens 6 flows, one public IP serves ≈10k concurrent page loads.

5.7 IPv6: More Addresses and a Simpler Header

Intuition first. IPv6 keeps IP's forwarding idea but removes the two crutches—private addresses plus NAT, and router fragmentation—by making addresses plentiful and headers router-friendly. Addresses are 128-bit ($2^{128} \approx 3.4 \times 10^{38}$), written as eight hex groups, e.g. `2001:db8:0:0:0:0:0:1`, compressible to `2001:db8::1` (one `::` run, leading zeros droppable). A typical home gets a /64 (as many addresses as the whole IPv4 Internet squared per subnet); key scopes are global unicast `2000::/3`, link-local `fe80::/10` (single link, like ARP scope), loopback `::1`, and multicast `ff00::/8`. Subnetting works as in IPv4: /64 network + 64-bit interface ID, often auto-configured (SLAAC) from the MAC. The header is fixed 40 B (vs. variable 20–60 B): version, traffic class, 20-bit flow label (QoS flows without deep inspection), payload length, next-header (replaces protocol), hop limit (renamed TTL), and 16 B source/destination. Dropped: header checksum (links already check), IHL/options (moved to chained extension headers: hop-by-hop, routing, fragment, ESP/AH, only processed when present), and router fragmentation (only the source fragments, after Path MTU Discovery; routers send ICMPv6 Packet-Too-Big). Fixed layout means fast hardware parsing; extension chaining keeps the common case cheap. *Worked micro-example.* Expand `2001:db8::7` to `2001:0db8:0000:0000:0000:0000:0000:0007` (eight groups). Under prefix `2001:db8:abcd::/48`, the first 48 bits are fixed, leaving 2^{80} addresses—enough for 2^{16} customer /64s each holding 2^{64} hosts. *Transition:* dual-stack hosts run both protocols (DNS `A` vs. `AAAA` selects); tunnels (6to4, Teredo) carry v6 over v4; NAT64/DNS64 lets v6-only clients reach v4 servers. Adoption was slow because NAT made v4 "good enough" and every middlebox must upgrade—but mobile and cloud growth now push v6 past half of Google traffic.

5.8 Putting It Together: A Packet's Life Through the Network Layer

Trace a ping from NTU host `10.1.2.3/24` to `203.0.113.10/24` via two routers. Host checks destination: same prefix? No (10.1.2.0/24 $\neq$ 203.0.113.0/24), so it forwards to its gateway `10.1.2.1`. It ARPs for the gateway MAC, wraps the IP datagram in an Ethernet frame, and sends. Router 1 strips the frame, does LPM on `203.0.113.10`: longest match is `203.0.113.0/24` $\rightarrow$ link 1. It decrements TTL, recomputes checksum, ARPs for next-hop MAC if needed, and re-encapsulates. Router 2 repeats LPM and delivers to the subnet; the destination host strips headers and replies. This hop-by-hop re-wrapping is why IP can run over any link technology.

Remark 5.1 (Why best-effort?). IP deliberately does not retransmit or order packets. Reliability is left to TCP at the edges. This keeps routers simple and fast—they just forward. Complexity at the edges, simplicity in the core is the end-to-end principle that scaled the Internet. If IP did reliability, every router would need per-flow state and timers, exploding cost.

A second example: Path MTU Discovery. Host sends with DF=1 and large size; a router with MTU 1280 drops it and returns ICMP "Packet Too Big" with its MTU. Host lowers size and retries. No fragmentation occurs, yet delivery succeeds—cooperation between network (ICMP) and transport/host improves efficiency.

Aggregation examples: two /24s aggregate to a /23 (512 addresses), two /23s to a /22 (1024), and ISP block exttt14.24.0.0/15 holds 131 072. This hierarchy keeps the global BGP table tractable. This aggregation principle is applied recursively: RIRs allocate large blocks to ISPs, ISPs sub-allocate to customers, and longest-prefix match ensures the most specific advertisement wins. Without it, the global table would exceed a million entries and exceed router memory.

A practical check: run `ip route show` on Linux or `show ip bgp summary` on a Cisco router. You will see prefixes, next-hops, and AS-paths exactly as described.

5.9 Chapter Notes

IPv4 details in RFC 791, CIDR in RFC 4632, OSPF in RFC 2328, BGP-4 in RFC 4271. Kurose–Ross Ch. 4 and Peterson–Davie give fuller case studies. Dijkstra (1959) and Bellman–Ford (1958) are covered algorithmically in SC2001; here we use them as routing protocols. For hands-on, run `traceroute` to see TTL in action and `ip route` to inspect a forwarding table.

5.10 Exercises

E5.1 **CIDR and table size.** An ISP owns 14.24.0.0/15. (a) How many /24 blocks does it contain? (b) If it previously advertised each /24, how many BGP entries does aggregation save? (c) A customer needs 60 hosts; what smallest prefix length suffices and what mask? (d) Why does aggregation not always reduce table size?

E5.2 **Longest-prefix match by hand.** Table: 10.0.0.0/8 $\rightarrow$ 0, 10.1.0.0/16 $\rightarrow$ 1, 10.1.1.0/24 $\rightarrow$ 2, 0/0 $\rightarrow$ 3. For dst 10.1.1.23, 10.1.2.5, 192.0.2.1 give egress link. Draw the binary trie and show how LPM walks it.

E5.3 **Dijkstra trace and proof.** Run Dijkstra from s on Fig. 5.2 to completion; list extraction order and final distances to t. Prove the invariant: when u leaves the PQ, $dist[u] = \delta(s, u)$. Where does $w \geq 0$ get used? Give a counterexample with a negative edge.

E5.4 **Count-to-infinity.** Three nodes a–b–c line ($c = 1$ each). Link b–c cost $\to \infty$ (failure). Show distance-vector updates without poisoned reverse and count steps to converge (assume infinity = 16). Then show poisoned reverse fixes this case and give a 3-node loop where it still fails.

E5.5 **BGP policy vs. shortest path.** AS2 learns 192.0.2.0/24 via AS3 (customer, path [3]) and via AS4 (peer, path [4,3]). Which does BGP prefer under standard customer>peer policy? If AS2 forwards via hot-potato, how does internal OSPF cost affect egress choice? Sketch the full decision steps (local-pref, AS-path, MED, IGP cost) and when each applies.

Chapter 6

Link Layer: Ethernet, MAC, ARP and Switching

> *"The link delivers frames to neighbours; the network delivers packets to the world."* — Every IP packet rides inside a link-layer frame for one hop at a time.

From zero: we build here, no prior framing assumed. We start from "how do two machines share a wire?" with pictures before headers. Every notion—frame, MAC address, ARP, CSMA/CD, switch table, VLAN—is defined on first use and linked to the IP layer above. No link-layer background is assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) explain link-layer services: framing, error detection, and reliable delivery;

(L2) distinguish MAC from IP addresses and describe ARP resolution and caching;

(L3) describe Ethernet framing, addressing, and CSMA/CD with exponential backoff;

(L4) explain how switches self-learn, forward, filter, and support VLANs;

(L5) contrast hubs, switches, and routers in terms of collision vs. broadcast domains.

6.1 What the Link Layer Does

The network layer hands a datagram to the link layer, which must move it *one hop* to the next node. Think of a courier van between two adjacent post offices: it wraps the envelope (IP packet) in a van manifest (frame header/trailer), drives it one segment, unwraps at the next office. *Services:* framing (delimit where frame starts/ends), error detection (CRC), optional reliable delivery (rare on wired links—left to TCP), flow control, and medium access (who talks when). Framing is non-trivial: the receiver must find frame boundaries in a

continuous bit stream. Ethernet uses preamble + length/Type; HDLC uses bit stuffing with flags `01111110`. Key idea: *IP addresses are end-to-end (unchanged); MAC addresses are hop-by-hop (rewritten by each router).*

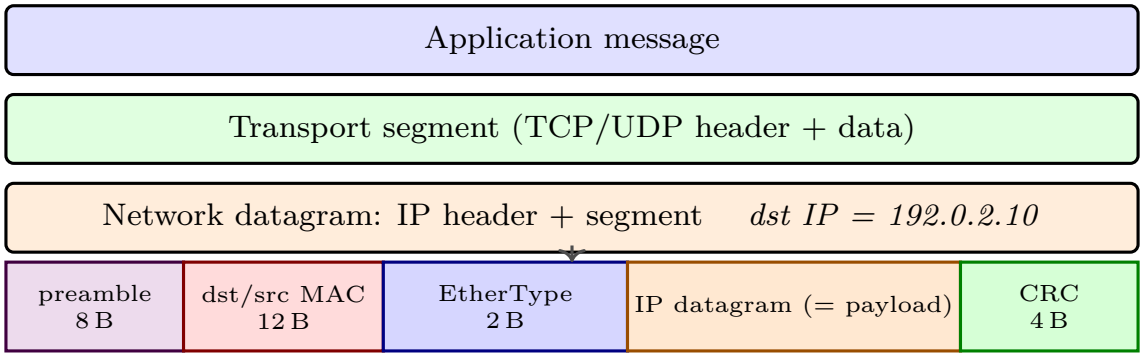


Figure 6.1: Encapsulation: IP datagram becomes the payload of an Ethernet frame. Each hop re-wraps with new MAC addresses.

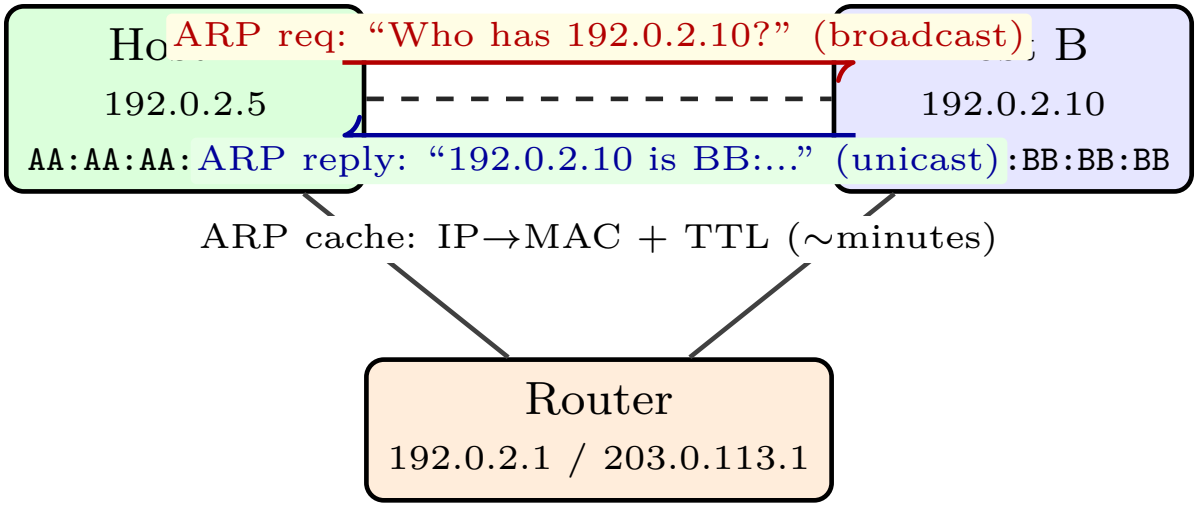
A router strips the incoming frame's MAC header, looks up the IP, and builds a new frame with its own MAC as source and next-hop's MAC as destination. Encapsulation overhead matters: Ethernet adds 18 B header/trailer + 8 B preamble, so a 1500 B IP packet becomes 1526 B on wire (plus inter-frame gap 12 B). Error detection: parity catches odd bit flips; Internet checksum catches many bursts; *CRC-32* (Ethernet) divides the bit stream by a generator polynomial and appends the remainder—detects all bursts < 32 bits and any odd number of errors. Bad CRC $\rightarrow$ drop; no retransmission at link layer on wired links (wireless 802.11 does retransmit).

6.2 MAC Addresses and ARP

A *MAC address* (Media Access Control) is a 48-bit identifier burned into each network interface (NIC), written as hex: `1A:2B:3C:4D:5E:6F`. Uniqueness via OUI (first 24 bits = vendor). Broadcast address is `FF:FF:FF:FF:FF:FF`. Unlike IP, MAC is flat—no hierarchy, no aggregation, hence not routable beyond one LAN. IP and MAC solve different problems: IP is hierarchical and routable (like postal code); MAC is flat and local (like serial number on the van). To send an IP datagram on a link, the sender must know the *next-hop MAC*. *ARP* (Address Resolution Protocol) maps IP $\rightarrow$ MAC within a subnet. Hosts also use DHCP to *obtain* their own IP; ARP then resolves neighbours.

Definition 6.1 (ARP cache)**.** Each host/router maintains a table IP $\mapsto$ (MAC, TTL). On miss, it broadcasts an ARP request; the owner replies. Entries expire (typically 2–20 min) to handle NIC changes. Gratuitous ARP (announce own mapping) detects duplicate IPs.

```python
# Simplified ARP cache with TTL
import time
arp_cache = {}  # ip_str -> (mac_str, expiry)
TTL = 300  # seconds
def arp_lookup(ip):
    entry = arp_cache.get(ip)
    if entry and entry[1] > time.time():
        return entry[0]
    return None  # -> broadcast ARP request
```



If dst is off-subnet, ARP for gateway MAC (router), not distant host.

Figure 6.2: ARP in a subnet: A broadcasts a request; B unicasts a reply. Both cache the mapping to avoid future broadcasts.

```
10
11 def arp_update(ip, mac):
12     arp_cache[ip] = (mac, time.time() + TTL)
13
14 def send_datagram(dst_ip, payload):
15     mac = arp_lookup(dst_ip)
16     if mac is None:
17         print(f"ARP broadcast: who has {dst_ip}?")
18         # ... wait for reply, then arp_update(dst_ip, reply_mac)
19     # build Ethernet frame with dst MAC and send
```

If destination is outside the subnet, the sender ARPs for the *gateway* (first-hop router) MAC and sends the frame there; the router forwards via IP. This is why “default gateway” is configured—it is the next-hop MAC for all off-subnet traffic. ARP spoofing (forged replies) enables MITM; static entries and Dynamic ARP Inspection mitigate it.

6.3 Ethernet and CSMA/CD

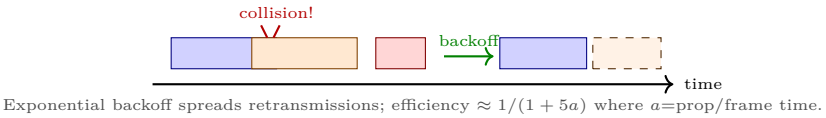
Ethernet (IEEE 802.3) is the dominant wired link. Frame format (excluding preamble): dst MAC (6 B) | src MAC (6 B) | EtherType (2 B) | payload (46–1500 B) | CRC (4 B). Payload <46 B is padded to meet 64 B minimum (collision detection needs a minimum frame time of 64 B × 8 / rate). EtherType says what is inside: `0x0800`=IPv4, `0x86DD`=IPv6, `0x0806`=ARP. Medium access: classic Ethernet is a *broadcast medium* (coax, or hub). Who may transmit? *CSMA/CD* (Carrier Sense Multiple Access with Collision Detection):

(a) Carrier sense: listen; if idle, transmit; if busy, wait.

(b) Collision detection: while transmitting, listen; if collision (voltage spike), abort and jam.

(c) Exponential backoff: after kth collision, pick random $r \in \{0, \ldots, 2^k - 1\}$ (capped at $k = 10$), wait $r \times 512$

bit-times, retry ($k \le 16$ else drop).

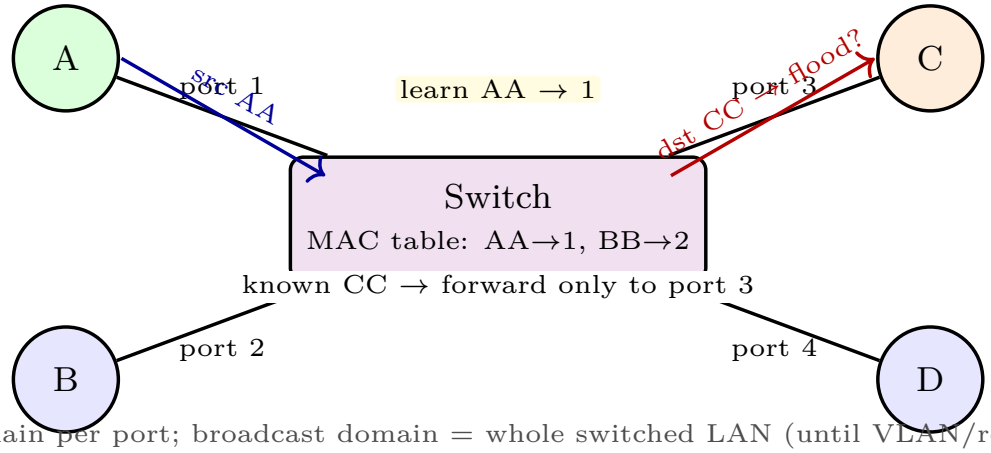


Exponential backoff spreads retransmissions; efficiency $\approx 1/(1+5a)$ where a=prop/frame time.

Modern switched Ethernet is full-duplex with no collisions—CSMA/CD remains only as a fallback and for Wi-Fi's cousin CSMA/CA (which avoids rather than detects collisions). Efficiency intuition: a= propagation / transmission time; if a is small, channel is idle quickly and efficiency is high. Switched links with $a \to 0$ approach 100% utilisation. CRC-32 detects bit errors; bad frames are dropped (no retransmission at link layer—TCP will handle it). A 10 Gbps link with BER 10^{-12} still sees a bad frame every few seconds at full rate, so CRC is essential.

6.4 Switches, Learning, and VLANs

A *hub* repeats bits to all ports (one collision domain). A *switch* is smarter: it has a *forwarding table* mapping MAC → port, built by *self-learning*. Unlike a router (IP table via routing protocols), a switch needs no configuration. *Learning:* when a frame arrives on port p with source MAC s, record $(s \to p)$. *Forwarding:* look up destination MAC d: if found and $p' \neq p$, forward only to p' (filtering); if unknown, flood to all ports except p; if $p' = p$, drop (same segment). Entries age out (∼5 min). No configuration needed—plug and play.



Collision domain per port; broadcast domain = whole switched LAN (until VLAN/router cuts it).

Figure 6.3: Self-learning switch: learns source MACs, forwards by destination lookup. Unknown destinations are flooded; learning makes future frames unicast.

```python
class Switch:
    def __init__(self, n_ports): self.table={}; self.n=n_ports
    def receive(self, frame, in_port):
        self.table[frame.src] = in_port          # learn source
        out = self.table.get(frame.dst)
        if out is None:          # unknown -> flood
            return [p for p in range(self.n) if p!=in_port]
        elif out == in_port:     # same segment -> filter
            return []
        else:                    # known -> unicast forward
```

```
            return [out]

# Example: A->B then B->A
sw = Switch(4)
print(sw.receive(type('F',(),{'src':'AA','dst':'BB'})(),0))  # flood [1,2,3]
print(sw.table)  # {'AA':0}
print(sw.receive(type('F',(),{'src':'BB','dst':'AA'})(),1))  # [0]
```

Switches create *per-port collision domains* but still one *broadcast domain*. To isolate broadcast, use *VLANs* (IEEE 802.1Q): tag frames with 12-bit VLAN ID (4 B extra header between src MAC and EtherType), switch forwards only within the same VLAN—as if multiple virtual switches. A trunk port carries multiple VLANs tagged; an access port belongs to one VLAN untagged. Routers (or L3 switches) connect VLANs at layer 3. *Hub vs. switch vs. router:* hub = physical layer (repeats, one collision, one broadcast); switch = link layer (learns, many collisions isolated, one broadcast/VLAN); router = network layer (routes by IP, cuts both collision and broadcast domains). Loops between switches need Spanning Tree Protocol (STP) to prune to a tree; otherwise flooded frames loop forever.

6.5 Putting It Together: A Frame's Life One Hop

Follow a datagram from router R1 to host B across Ethernet. R1 has IP datagram for `192.0.2.10` and knows next-hop is directly connected on `eth0` (`192.0.2.1/24`). It checks ARP cache for `192.0.2.10`: miss, so it broadcasts `Who has 192.0.2.10?` B replies with its MAC; both cache the binding. R1 now builds the frame: dst MAC = B, src MAC = R1-eth0, EtherType = `0x0800`, payload = IP datagram, CRC appended. It transmits: on switched full-duplex, no CSMA/CD contention; on a hub, it would sense and back off. Switch S2 receives on port 2, learns `R1-MAC` $\rightarrow$ `2`, looks up `B-MAC`: if known on port 4, forwards only there; if unknown, floods. B's NIC checks dst MAC matches (or broadcast), verifies CRC, strips header, hands IP datagram to Network layer. On the reverse path, B will already have R1's MAC cached, so no ARP is needed—one broadcast serves many packets until TTL expiry.

Aggregation examples: two /24s aggregate to a /23 (512 addresses), two /23s to a /22 (1024), and ISP block exttt14.24.0.0/15 holds 131 072. This hierarchy keeps the global BGP table tractable.

Remark 6.1 (Why no IP routing on a switch?). A switch never looks at the IP address; it forwards by MAC table built from observing sources. This makes it plug-and-play but also broadcast-heavy. A router, by contrast, forwards by IP prefix and cuts broadcasts. That is why large LANs are segmented into VLANs (L2 isolation) connected by routers (L3).

Error case: CRC fails (bit error on wire). The switch or host drops the frame silently. No link-layer retransmission on wired Ethernet—IP also does not retransmit. TCP at the endpoints will timeout and retransmit, illustrating layering: reliability only where it is end-to-end.

A quick lab check: run `arp -a` to see cached bindings, `ip link` to see MACs, and capture with Wireshark to inspect Ethernet II headers and the EtherType field. Compare the MAC addresses hop-by-hop with the unchanged IP addresses.

Spanning Tree note: with redundant switches, STP elects a root, blocks one port to break the loop, and unblocks on failure. Without STP, a single broadcast frame would loop forever and multiply exponentially

(broadcast storm).

6.6 Chapter Notes

Ethernet v2 / IEEE 802.3, ARP in RFC 826, VLANs in 802.1Q, STP in 802.1D. Kurose–Ross Ch. 6 and Peterson–Davie cover Ethernet evolution from 10 Mbps coax to 400 Gbps switched full-duplex. Wireshark captures make headers concrete—try `arp -a` and inspect `Ethernet II` headers in any capture.

6.7 Exercises

E6.1 **MAC vs. IP.** Host A (IP_A, MAC_A) sends to B (IP_B, MAC_B) via router R (MAC_R left, MAC'_R right). List src/dst IP and MAC at each hop for A→R→B. Which addresses change and which do not? Why must MAC change?

E6.2 **ARP cache.** A pings B for the first time in a /24. Describe each ARP message, who broadcasts, who replies, and what each cache contains after. What happens on the second ping? When would an entry expire or be overwritten?

E6.3 **Ethernet overhead.** A 40 B TCP ACK (20 B IP + 20 B TCP) is sent over Ethernet. What is the on-wire frame size (with padding, headers, preamble, IFG)? What fraction is overhead? Repeat for 1500 B payload and comment.

E6.4 **Switch learning.** Switch with 4 ports, initially empty table. Frames arrive: A(src AA, dst BB, port1), B(src BB, dst AA, port2), C(src CC, dst DD, port3). Show table after each and which ports each frame is forwarded to. What if D replies (src DD, dst CC, port4)? What if C sends to AA?

E6.5 **CSMA/CD backoff.** After $k = 3$ collisions, what is the contention window size? If A picks 2 and B picks 5 (slot = 51.2 μs for 10 Mbps), how long does each wait? Why does capping at $k = 10$ and $k_{\max} = 16$ bound unfairness and latency? Compare with CSMA/CA in Wi-Fi.

Chapter 7

Wireless and Mobile Networks: 802.11 and Beyond

"Wires tell you where the network is; wireless tells you where it could fail." — From fighting fading and hidden terminals to handing off a phone call at 300 km/h, this chapter makes wireless concrete: why CSMA/CD does not work in air, how Wi-Fi dodges collisions without detecting them, and how cellular keeps you connected while moving.

From zero: we build here, no prior wireless knowledge needed. We start from intuition—shouting across a noisy room—then formalise: SNR, path loss, and medium access. Every 802.11 and cellular idea is introduced as picture → words → definition, then tied to the IP stack you already know.

Learning Outcomes

After this chapter you will be able to:

(L1) explain why wireless links differ from wired: fading, hidden terminals, and bit errors;

(L2) describe 802.11 architecture (BSS, ESS, AP, channels) and why association matters;

(L3) analyse CSMA/CA with DIFS/SIFS, backoff, RTS/CTS and NAV;

(L4) sketch the 802.11 frame and explain rate adaptation and mobility handoff;

(L5) outline cellular evolution (2G/3G/4G/5G) and Mobile IP triangle routing.

7.1 Why Wireless Is Hard: Intuition First

Imagine two people shouting across a courtyard with a wall between them. They cannot hear if the other is already shouting (no collision detection), the wall attenuates sound (fading), and a third person near each listener hears differently (hidden terminal). Wireless is exactly this: a shared, noisy, distance-dependent broadcast medium.

Formally, received power follows path loss $P_r \propto P_t/d^\alpha$ with $\alpha \in [2,4]$, plus shadowing and multipath fading that makes P_r vary on millisecond scales. Signal-to-noise ratio $\text{SNR} = P_r/N_0$ sets bit-error rate (BER); higher SNR → lower BER → more bits per symbol (e.g., BPSK → 64-QAM). A *hidden terminal* is A–B–C where A and C cannot hear each other but both can reach B; an *exposed terminal* is $B \to A$ blocking $C \to D$ though the two transfers would not collide.

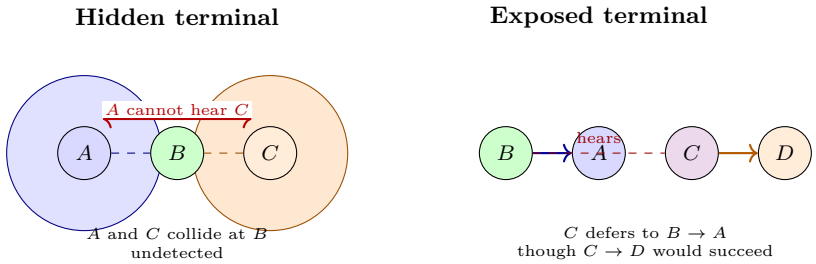


Figure 7.1: Left: hidden terminal — A and C hidden from each other collide at B. Right: exposed terminal — C is needlessly blocked. Both motivate RTS/CTS and carrier sensing.

Wired Ethernet can detect collisions while transmitting (CD) because the signal on the wire is observable. On wireless, a sender cannot listen while sending (its own power drowns others) and not all senders hear each other. So 802.11 avoids collisions rather than detecting them.

7.2 802.11 Architecture: BSS, ESS, and Channels

The basic unit is the *Basic Service Set (BSS)*: one Access Point (AP) plus its associated stations. In *infrastructure* mode every frame goes through the AP, even $A \to B$ within the same BSS. An *Extended Service Set (ESS)* links multiple BSSs via a distribution system (usually wired Ethernet), sharing one SSID. *Ad hoc* (IBSS) has no AP; stations talk peer-to-peer.

Each BSS operates on a channel. At 2.4 GHz there are 11 (US) overlapping channels; only 1, 6, 11 are non-overlapping (22 MHz each). At 5 GHz, 20/40/80 MHz channels offer many non-overlapping options and higher rates. An AP chooses a channel to minimise interference; clients scan (active probe or passive beacon listen) then *associate* via authentication and association frames. The AP maintains an association table mapping station MAC → BSS.

mobility within an ESS requires handoff: the station reassociates to a new AP; the distribution system updates forwarding. We revisit handoff formally in §7.5.

7.3 Medium Access: CSMA/CA in Depth

Carrier Sense Multiple Access with Collision Avoidance is the heart of Wi-Fi. Rules:

1. Sense channel: if idle for DIFS (e.g., 50 μs), transmit. If busy, defer and enter backoff.
2. Backoff: pick uniform $k \in [0, \text{CW}]$, wait $k \times$SlotTime while channel idle; freeze timer when busy; resume when idle for DIFS. CW starts at $\text{CW}_{\min}$ (15) and doubles on collision up to $\text{CW}_{\max}$ (1023).
3. Receiver waits SIFS (10 μs) then sends ACK. SIFS < DIFS so ACK wins over new data.

4. Optional RTS/CTS: sender sends short RTS, receiver replies CTS; both carry duration in NAV (network allocation vector) so neighbours that hear either frame defer, solving hidden terminals. Overhead makes RTS/CTS worthwhile only for large frames.

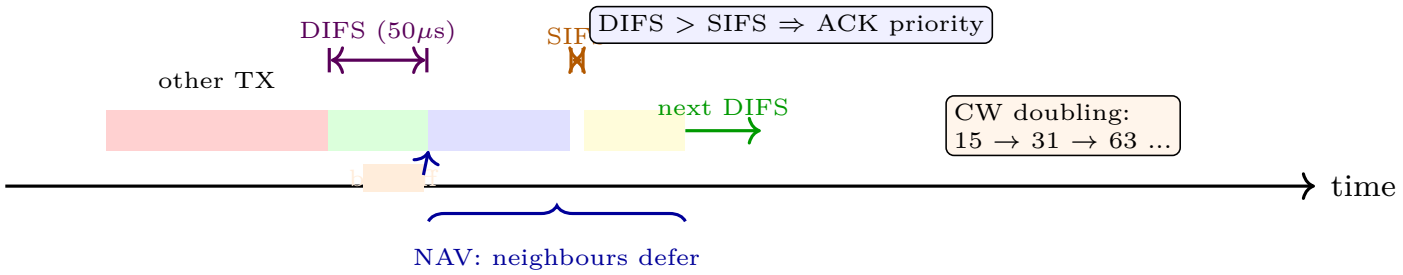


Figure 7.2: CSMA/CA timing: wait DIFS, count down backoff only while idle, transmit, wait SIFS for ACK. NAV from RTS/CTS reserves the channel at neighbours.

```python
import random
CW_min, CW_max, SLOT = 15, 1023, 9  # slots, microsec
def csma_ca_transmit(channel_busy_fn, max_retries=6):
    CW = CW_min
    for attempt in range(max_retries+1):
        # 1. wait for DIFS idle (simplified: poll)
        while channel_busy_fn(): pass
        # DIFS elapsed -- start backoff
        k = random.randint(0, CW)
        while k > 0:
            if channel_busy_fn():  # freeze
                while channel_busy_fn(): pass  # wait DIFS again
            else:
                k -= 1  # one idle slot
        # 2. transmit and wait for ACK (SIFS later)
        ack = send_frame_and_wait_ack(timeout_us=50)
        if ack:
            return True
        CW = min(2*CW + 1, CW_max)  # binary exponential backoff
    return False

def snr_to_rate(snr_db):
    # toy rate adaptation: pick modulation by SNR threshold
    if snr_db > 25: return "64-QAM 54 Mbps"
    elif snr_db > 18: return "16-QAM 24 Mbps"
    elif snr_db > 10: return "QPSK 12 Mbps"
    else: return "BPSK 6 Mbps"
```

Why not CSMA/CD? Two reasons already seen: the sender cannot detect a collision at the receiver (hidden terminal), and it cannot listen while transmitting. CA plus ACK is the workaround: absence of ACK implies collision, triggering CW doubling exactly as Ethernet does after CD.

7.4 Frames, Rate Adaptation, and Power

Each 802.11 data frame carries: Frame Control (2 B), Duration/NAV (2 B), three addresses (receiver, transmitter, BSS), sequence number, payload (0–2312 B), and FCS/CRC (4 B). Address 3 distinguishes BSS traffic via AP.

Rate adaptation picks modulation/coding per SNR and BER. High SNR → dense constellations (64-QAM, 256-QAM) at tens to hundreds of Mbps; low SNR → robust BPSK. Minstrel and similar algorithms probe rates and track ACK success. *Power management*: stations sleep between beacons; the AP buffers frames and announces them in TIM (traffic indication map); stations poll with PS-Poll.

7.5 Mobility and Cellular

Mobility has two facets: staying connected while moving within one network (handoff) and being reachable at a new address (Mobile IP / cellular).

Wi-Fi handoff: scanning (active/passive), authentication, reassociation; the ESS distribution system learns the new AP. Hard handoff briefly disconnects; 802.11r fast transition pre-authenticates.

Mobile IP solves reachability: a mobile node has a permanent home address and a temporary care-of address (CoA) at the foreign network. A home agent intercepts packets to the home address and tunnels them to the CoA (triangle routing); route optimisation can shortcut later. Registration via the foreign agent keeps the binding current.

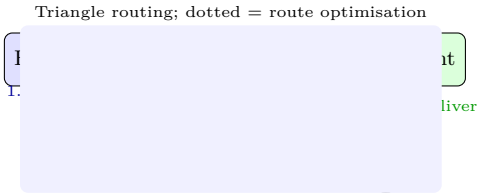


Figure 7.3: Mobile IP: correspondent (CN) sends to home address; home agent tunnels to foreign agent / care-of address; MN can reply directly when optimised.

Cellular evolution: 2G (GSM, digital voice), 3G (UMTS, Mbps data), 4G LTE (all-IP, OFDMA, 100 Mbps, EPC core with MME/S-GW/P-GW, eNodeB base stations), 5G (mmWave + sub-6 GHz, massive MIMO, beamforming, network slicing, ultra-low latency via edge, gNodeB and service-based core). In all generations a base station covers a cell; handoff is network-controlled and make-before-break to keep calls alive.

7.6 Throughput, Fairness, and Why Wi-Fi Feels Slower

Raw rate is not goodput. Each frame pays DIFS + backoff + preamble + header + SIFS + ACK. At 54 Mbps, a 1500 B frame takes 222 μs on air but $\approx 50 + 67 + 222 + 10 + 30 \approx 379\,\mu$s with overhead, so single-station goodput ≈ 31.6 Mbps (58%). With n contending stations, collisions and larger CW further reduce it.

Example: two stations saturated, $\mathrm{CW}_{\min} = 15$. Average backoff ≈ 7.5 slots $= 67\,\mu$s. Collision probability $p \approx 1/(1 + \mathrm{CW}) \approx 6\%$ with two stations; on collision CW doubles and retries add delay. Aggregate goodput

drops to 20–25 Mbps. Capture effect and rate heterogeneity worsen fairness: a slow 6 Mbps station occupying air 9× longer than a 54 Mbps station drags everyone (performance anomaly); 802.11e TXOP and airtime fairness mitigate by granting equal airtime, not equal packets.

Power and scanning cost matter for phones: passive scanning across 11 channels at 100 ms per channel costs > 1 s to discover APs; active probing cuts this to tens of ms but burns energy. Hence phones prefer to stay associated and use 802.11k/v neighbour reports to prune scan lists.

7.6.1 Worked Example: Goodput in Numbers

Take 802.11g at 54 Mbps, slot 9 μs, DIFS 28 μs, SIFS 10 μs, preamble 20 μs, ACK 14 B at 24 Mbps $\approx 30\,\mu$s. For a 1500 B payload:

- Data airtime: $(1500 \times 8)/54\text{Mbps} \approx 222\,\mu$s plus preamble $20\,\mu\text{s} = 242\,\mu$s.
- Overhead before data: DIFS $28\,\mu$s + average backoff $7.5 \times 9 = 67.5\,\mu$s.
- Overhead after data: SIFS $10\,\mu$s + ACK $30\,\mu$s.
- Total per frame $\approx 28 + 67.5 + 242 + 10 + 30 = 377.5\,\mu$s, goodput $\approx 1500 \times 8/377.5\,\mu\text{s} \approx 31.8$ Mbps.

With three saturated stations collision rate rises; simulation with CW doubling shows aggregate goodput falls to ≈ 22 Mbps, so per-station share is ≈ 7 Mbps despite 54 Mbps PHY rate.

7.7 Wireless Security Glimpse: WEP → WPA2/3

WEP (Wired Equivalent Privacy) failed because it reused RC4 keystreams with a 24-bit IV: after ≈ 5000 packets an IV repeats, and with enough repeats the keystream is recovered; plus CRC32 is not a MAC, so bit-flipping goes undetected. WPA2 fixes both: AES-CCMP provides confidentiality and integrity with a 48-bit packet number (PN) that never repeats within a key lifetime, and 802.1X/EAP does mutual authentication via a 4-way handshake deriving a fresh PTK per session. WPA3 adds forward secrecy (SAE handshake) and opportunistic encryption. Details of crypto, MACs and handshakes are in Ch. 8.

7.8 Chapter Notes

802.11 standards are IEEE 802.11a/b/g/n/ac/ax (Wi-Fi 4/5/6); cellular specs are 3GPP LTE/NR. Tanenbaum & Wetherall and Kurose & Ross cover wireless chapters with matching figures; the hidden/exposed duality and NAV are the exam favourites. Next chapter builds the crypto that makes WPA2/3 and TLS possible.

7.9 Exercises

E7.1 **Hidden vs. exposed.** Draw A–B–C–D line topology with ranges $A \leftrightarrow B$, $B \leftrightarrow A, C$, $C \leftrightarrow B, D$, $D \leftrightarrow C$. Identify hidden and exposed pairs and state for each whether RTS/CTS helps, hurts, or is

neutral; quantify overhead if RTS/CTS = 20 B and data = 1500 B at 54 Mbps.

E7.2 **CSMA/CA walk-through.** Two stations both wait DIFS and pick backoff 2 and 5 (CW=7, slot 9 μs). Tabulate countdown slot by slot including freeze when the first transmits, compute total idle time before the second starts, and show CW evolution if the second collides twice then succeeds.

E7.3 **Channel planning.** In 2.4 GHz with channels 1/6/11, place 4 APs on a floor to minimise co-channel interference. Explain why 5 GHz with 24 non-overlapping 20 MHz channels changes the plan, and compute max aggregate throughput if each AP does 150 Mbps.

E7.4 **Rate vs. range.** Using the SNR thresholds in the listing, a station moves from SNR 28 dB to 12 dB. What rate is chosen before/after? If BER at 64-QAM is 10^{-3} at 12 dB but 10^{-5} at BPSK, estimate retransmissions per 1000 frames and effective goodput.

E7.5 **Mobile IP triangle.** CN sends 10 packets to MN's home address while MN is away. Trace each packet's path with and without route optimisation, count hops and tunnel overhead, and explain why ingress filtering can break naive direct replies from MN.

Chapter 8

Network Security: Crypto, TLS, and Firewalls

> *"Security is not a feature you add; it is a property you prove."* — Every packet you send may cross a hostile network. This chapter builds defence from zero: what crypto guarantees, how TLS stitches those guarantees into HTTPS, and where firewalls and VPNs fit when crypto alone is not enough.

From zero: we build here, no crypto background assumed. We start with intuition—locked boxes and seal wax—then formalise: confidentiality, integrity, authentication. Each primitive is picture → words → definition, with Python you can run to see it fail when misused.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish confidentiality, integrity, authentication and non-repudiation;

(L2) compare symmetric vs. public-key crypto and explain key distribution;

(L3) use hashes, MACs and digital signatures to detect tampering;

(L4) walk through a TLS 1.2/1.3 handshake and explain certificates;

(L5) place firewalls, IDS and VPNs in a layered defence and name their limits.

8.1 What Security Means: CIA and Threat Model

Picture sending a letter: you want no one but the recipient to read it (confidentiality), no one to alter it in transit (integrity), the recipient to be sure it came from you (authentication), and to be unable to deny it later (non-repudiation). Networks add an attacker who can *eavesdrop*, *modify*, *inject*, and *replay* packets anywhere on the path.

We formalise with a threat model: Dolev-Yao attacker controls the network (reads, drops, reorders every packet) but cannot break crypto without the key. Goals are CIA plus availability. Attacks: sniffing (passive read), spoofing/IP spoofing, man-in-the-middle (MITM), replay, denial-of-service. Defence is layered: crypto at the ends, firewalls at the borders, monitoring everywhere.

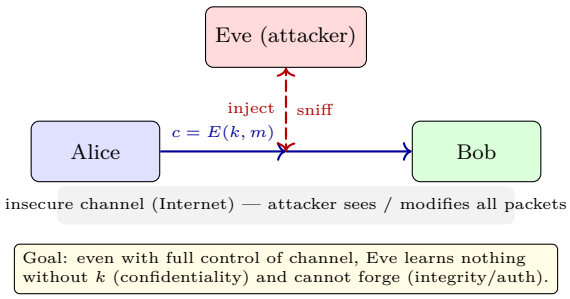


Figure 8.1: Threat model: Eve controls the channel between Alice and Bob. Crypto must protect m even though c is fully exposed.

8.2 Symmetric Cryptography

One key k locks and unlocks. Intuition: a single padlocked box where both sides share the key.

Formally, encryption $c = E(k, m)$, decryption $m = D(k, c)$ with $D(k, E(k, m)) = m$. Security means without k, c reveals nothing about m (indistinguishability). Two families:

Stream ciphers XOR a keystream $S(k, \text{nonce})$ with plaintext: $c = m \oplus S$. If nonce repeats, $c_1 \oplus c_2 = m_1 \oplus m_2$ leaks. **Block ciphers** (AES-128/256) encrypt fixed 128-bit blocks; longer messages need a mode: CBC, CTR, or GCM (which adds integrity). Never use ECB (identical plaintext blocks → identical ciphertext blocks).

Key size matters: AES-128 has 2^{128} keys; brute force is infeasible, but a 40-bit key falls in hours. Key distribution is the pain: how do Alice and Bob share k without Eve reading it? That needs public-key crypto.

```python
# Toy demo: AES-like usage via Python's toy XOR (do NOT use raw XOR in production!)
import os, hashlib
def xor_stream(key: bytes, nonce: bytes, msg: bytes) -> bytes:
    # keystream = SHA256(key||nonce||counter) repeated
    ks = b""
    for ctr in range((len(msg)+31)//32):
        ks += hashlib.sha256(key + nonce + ctr.to_bytes(4,'big')).digest()
    return bytes(a^b for a,b in zip(msg, ks[:len(msg)]))

key = os.urandom(16); nonce = os.urandom(8)
msg = b"Transfer $100 to Bob"
ct = xor_stream(key, nonce, msg)
pt = xor_stream(key, nonce, ct)  # decrypt = same XOR
assert pt == msg
# Reusing nonce leaks: ct1 ^ ct2 == msg1 ^ msg2
ct2 = xor_stream(key, nonce, b"Transfer $900 to Eve")
leak = bytes(a^b for a,b in zip(ct, ct2))  # == msg ^ msg2
```

Hashes and MACs are next; they do not encrypt but ensure nothing changed.

8.3 Hashes, MACs, and Signatures

A *cryptographic hash* $H(m)$ is a fixed-size fingerprint (e.g., SHA-256 $\rightarrow$ 256 bits) that is preimage-resistant (given h, cannot find m), second-preimage resistant, and collision-resistant. Even one-bit change flips $\approx$ half the output bits.

A *MAC* (message authentication code) adds a key: MAC $= H(k \parallel m)$ in toy form, real HMAC is $H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$. Receiver recomputes and compares; mismatch means tampering or wrong key. Hash alone does not authenticate: Eve can replace $(m, H(m))$ with $(m', H(m'))$.

Digital signatures give public verifiability and non-repudiation via public-key crypto: Alice signs with private key, anyone verifies with public key, but only Alice could have signed.

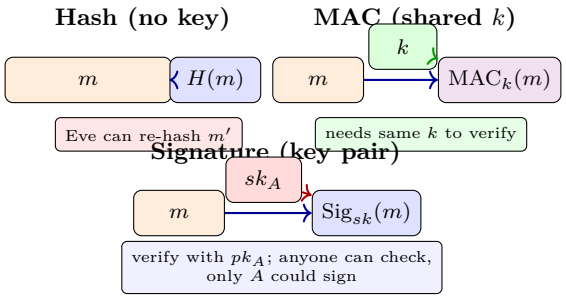


Figure 8.2: Hash detects accidental change; MAC detects tampering with a shared key; signature does it with a key pair and adds non-repudiation.

8.4 Public-Key Crypto and Key Exchange

Two keys: public pk (share widely) and private sk (never share). $c = E(pk, m)$, $m = D(sk, c)$. Analogy: an open padlock (pk) anyone can snap shut, but only you have the key (sk) to open.

RSA (Rivest-Shamir-Adleman): pick large primes p, q, $n = pq$, choose e coprime to $\phi(n) = (p-1)(q-1)$, compute $d = e^{-1} \bmod \phi(n)$. Then $pk = (e, n)$, $sk = (d, n)$, $E(m) = m^e \bmod n$, $D(c) = c^d \bmod n$. Security relies on factoring n being hard.

Diffie-Hellman lets Alice and Bob agree on a shared secret over a public channel: agree on prime p and generator g; Alice picks a, sends $A = g^a \bmod p$; Bob picks b, sends $B = g^b \bmod p$; both compute $s = B^a = A^b = g^{ab} \bmod p$. Eve sees A, B, g, p but not a, b, and computing g^{ab} is believed hard (discrete log).

```python
# Tiny RSA demo (toy primes -- real RSA uses 2048-bit primes!)
import math
p, q = 61, 53
n = p*q; phi = (p-1)*(q-1)
e = 17  # coprime to phi
d = pow(e, -1, phi)  # modular inverse (Python 3.8+)
print(f"pk=({e},{n}) sk=({d},{n})")
def rsa_encrypt(m, e, n): return pow(m, e, n)
def rsa_decrypt(c, d, n): return pow(c, d, n)
msg = 42
ct = rsa_encrypt(msg, e, n); pt = rsa_decrypt(ct, d, n)
assert pt == msg, "RSA round-trip failed"
```

```
print(f"msg {msg} -> ct {ct} -> pt {pt}")

# Diffie-Hellman (tiny p for demo)
p_dh, g = 23, 5
a, b = 6, 15
A = pow(g, a, p_dh); B = pow(g, b, p_dh)
s_alice = pow(B, a, p_dh); s_bob = pow(A, b, p_dh)
assert s_alice == s_bob
print(f"DH shared secret: {s_alice}")
```

RSA is slow ($\approx 1000\times$ slower than AES) so we use hybrid: RSA or DH to exchange a short AES key, then AES for bulk data. The signature direction is reversed: sign with sk, verify with pk.

8.5 TLS: How HTTPS Works

TLS stitches everything: authentication via certificates, key exchange via (EC)DHE or RSA, bulk encryption via AES/ChaCha, integrity via MAC/GCM. TLS 1.3 removed RSA key transport and static DH to enforce forward secrecy.

Simplified handshake (TLS 1.2/1.3 core):

1. ClientHello: client nonce r_C, offered ciphers, key share.
2. ServerHello: server nonce r_S, chosen cipher, key share, Certificate (chain to trusted CA), ServerKeyExchange if needed.
3. Client verifies certificate (CA signature, hostname, expiry, revocation), computes premaster, derives session keys via PRF/hash.
4. Both send Finished (MAC over transcript) to confirm no tampering or downgrade.

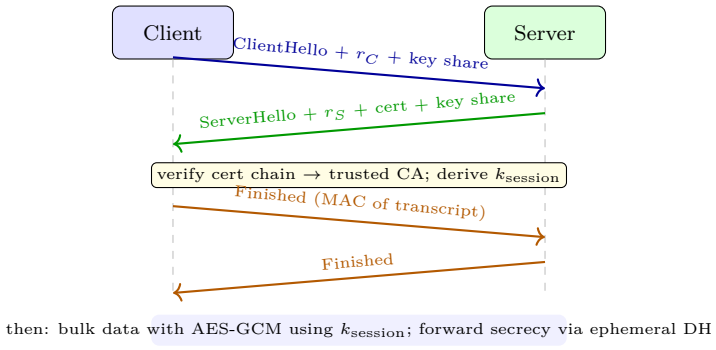


Figure 8.3: TLS handshake (simplified): nonces + key shares → session key, certificate authenticates server, Finished confirms integrity.

Now application data flows encrypted and integrity-protected. Certificates bind pk to a domain; a CA signs the binding. Browser ships ≈ 100 trusted CA roots; compromise of any CA breaks trust, hence Certificate Transparency logs.

8.6 Firewalls, IDS, and VPNs

Crypto protects data in transit; firewalls control *who* can connect at network borders.

Packet filter (stateless): rules on 5-tuple (src IP, dst IP, src port, dst port, protocol). Example: block inbound TCP SYN to port 22 except from 137.132.0.0/16. Cheap but cannot track connections.

Stateful filter: tracks TCP state (SYN, ESTABLISHED); allows inbound only if matching outbound established. Handles FTP active mode and tolerates ephemeral ports.

Application gateway / proxy: terminates connection, inspects application payload (e.g., HTTP filtering). Stronger but slower.

Placement: a *DMZ* between outer and inner firewalls hosts public servers (web, mail); inner firewall protects the private LAN. *IDS* (intrusion detection) sniffs and alerts on signatures or anomalies; *IPS* blocks inline. They do not replace crypto: a firewall cannot stop a phished password or a malicious insider; it just shrinks the reachable attack surface.

VPN (virtual private network) tunnels private traffic over the public Internet: IPSec at IP layer (tunnel vs. transport mode) or TLS-VPN at application layer. Site-to-site IPSec uses ESP (encrypted payload + integrity); remote-access VPN authenticates the user then assigns a virtual IP.

8.7 Putting It Together: Why TLS Alone Is Not Enough

TLS gives confidentiality and server authentication on one connection, but says nothing about what the server does with m afterwards (database breach, XSS, insider). Firewalls limit who reaches the TLS terminator, IDS spots brute-force and scans, and key management (rotation, HSM, least privilege) limits blast radius when a key leaks. Security is a chain: weakest link—often human (phishing, reuse of passwords, misconfigured S3 bucket)—breaks it regardless of cipher strength. Hence defence in depth: harden each layer and assume breach.

8.8 Chapter Notes

Schneier, Katz & Lindell, and Kurose & Ross Ch. 8 cover this material. For hands-on, try Wireshark to inspect a real TLS handshake and `iptables` / `nft` to write a stateful rule set; the gap between textbook and real config is where exams and interviews probe.

8.9 Exercises

E8.1 **CIA mapping.** For each scenario—(a) sniffing unencrypted HTTP, (b) altering DNS reply, (c) forging a grade-submission email—state which of confidentiality, integrity, authentication or non-repudiation fails and which crypto primitive (encryption, MAC, signature, cert) fixes it.

E8.2 **RSA by hand.** With $p = 5, q = 11, e = 3$ compute $n, \phi(n), d$ and encrypt $m = 6$ then decrypt. Show what goes wrong if you reuse k or use ECB mode for AES on an image (describe the visible artefact).

E8.3 **Hash vs. MAC vs. signature.** Alice sends $(m, H(m))$ without a key. Show Eve's exact forging steps. Then show why HMAC-SHA256 with a shared key stops forging but still allows Alice to deny sending m; explain how signing with sk_A fixes non-repudiation.

E8.4 **TLS trace.** List every TLS 1.2 message from ClientHello to Finished for a fresh connection to `example.com`. Mark which are encrypted, which prove possession of the server private key, and which provide forward secrecy when (EC)DHE is used vs. RSA key transport.

E8.5 **Firewall rules.** Write a stateful rule set: allow outbound HTTP/HTTPS/DNS, allow inbound only for established connections, block inbound SSH except from campus 137.132.0.0/16, and place a web server in a DMZ. Explain each rule and give a packet that is blocked and one that passes, plus what IDS would catch that the firewall misses.